

A Comprehensive Survey on Database Management System Fuzzing: Techniques, Taxonomy and Evaluation

XIYUE GAO, School of computer science and technology, Xidian University, Xi'an, China

ZHUANG LIU, School of Computer Science and Technology, Xidian University, Xi'an, China

JIANGTAO CUI, School of Computer Science and Technology, Xidian University, Xi'an, China

HUI LI, Xidian University, Xi'an, China and Shanghai Yunxi Technology Co., Ltd., Shanghai, China

HUI ZHANG, Shanghai Yunxi Technology Co., Ltd., Shanghai, China

KEWEI WEI, Shanghai Yunxi Technology Co., Ltd., Shanghai, China

KANKAN ZHAO, Shanghai Yunxi Technology Co., Ltd., Shanghai, China

Database Management System (DBMS) fuzzing is an automated testing technique aimed at detecting errors and vulnerabilities in DBMSs by generating, mutating, and executing test cases. It not only reduces the time and cost of manual testing but also enhances detection coverage, providing valuable assistance in developing commercial DBMSs. Existing fuzzing surveys mainly focus on general-purpose software. However, DBMSs are different from them in terms of internal structure, input/output, and test objectives, requiring specialized fuzzing strategies. Therefore, this article focuses on DBMS fuzzing and provides a comprehensive review and comparison of the methods in this field. We first introduce the fundamental concepts. Then, we systematically define a general fuzzing procedure and decompose and categorize existing methods. Furthermore, we classify existing methods from the testing objective perspective, covering various components in DBMSs. For representative works, more detailed descriptions are provided to analyze their strengths and limitations. Additionally, we review recent fuzzing methods for non-relational DBMSs as well as emerging techniques. To objectively evaluate the performance of each method, we present an open-source DBMS fuzzing toolkit, OpenDBFuzz. Based on this toolkit, we conduct a detailed experimental comparative analysis of existing methods and finally discuss future research directions.

CCS Concepts: • **Information systems** → **Database performance evaluation**; • **Software and its engineering** → *Software testing and debugging*;

Additional Key Words and Phrases: Automated database testing, fuzzing, DBMS fuzzing, DBMS fuzz testing, Experimental comparison

This work is supported by the National Natural Science Foundation of China under Grant No. 62302370, 62272369, and 62372352.

Authors' Contact Information: Xiyue Gao, School of computer science and technology, Xidian University, Xi'an, China; e-mail: xygao@xidian.edu.cn; Zhuang Liu, School of Computer Science and Technology, Xidian University, Xi'an, China; e-mail: zliu_01@stu.xidian.edu.cn; Jiangtao Cui, School of Computer Science and Technology, Xidian University, Xi'an, China; e-mail: cuijt@xidian.edu.cn; Hui Li (corresponding author), Xidian University, Xi'an, China and Shanghai Yunxi Technology Co., Ltd., Shanghai, China; e-mail: hli@xidian.edu.cn; Hui Zhang, Shanghai Yunxi Technology Co., Ltd., Shanghai, China; e-mail: zhanghui@inspur.com; Kewei Wei, Shanghai Yunxi Technology Co., Ltd., Shanghai, China; e-mail: weikewei@inspur.com; Kankan Zhao, Shanghai Yunxi Technology Co., Ltd., Shanghai, China; e-mail: zhaokankan@inspur.com.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2026 Copyright held by the owner/author(s).

ACM 0360-0300/2026/04-ART268

<https://doi.org/10.1145/3799227>

ACM Reference Format:

Xiyue Gao, Zhuang Liu, Jiangtao Cui, Hui Li, Hui Zhang, Kewei Wei, and Kankan Zhao. 2026. A Comprehensive Survey on Database Management System Fuzzing: Techniques, Taxonomy and Evaluation. *ACM Comput. Surv.* 58, 10, Article 268 (April 2026), 36 pages. <https://doi.org/10.1145/3799227>

1 Introduction

Database Management System (DBMS), as the fundamental software for managing and organizing data, is widely used in almost all software applications [2]. Its reliability and security are of paramount importance. DBMS testing aims to verify its functionality, performance, and security to ensure that it meets the expected requirements and standards [16, 60, 73]. After discovering potential errors and vulnerabilities, developers can quickly address and optimize them, which is crucial for the development of commercial DBMSs [8, 9].

Traditional DBMS testing heavily relies on manual testing, which is not only time-consuming and laborious but also suffers from a limited testing scope. To improve efficiency and accuracy, automated testing techniques have gradually emerged [17]. One such technique is DBMS fuzzing, which discovers DBMS bugs by automatically generating, mutating, and executing test cases [46]. The goal of fuzzing is to introduce a large amount of invalid, abnormal, or random data to test the robustness and security of DBMSs against adverse inputs [3, 5–7]. This approach significantly reduces the time and cost of manual testing while improving the coverage of bug detection.

As shown in Figure 1, the study of DBMS fuzzing has a long history. In 1978, Howden et al. first introduced the concept of a test oracle, which is a source of information about whether the output of a system is correct or not [46]. Since then, many types of methods have been proposed to construct a test oracle. In 1998, McKeeman introduced the concept of differential testing [78], and Chen et al. proposed metamorphic testing [24]. In the same year, SQL Server first applied fuzzing to the database field [95], which generated a large number of statements based on the rules and conducted differential tests on different DBMSs. In 2007, the DBMS differential testing method proposed by Bati et al. adopted a mutation-based generator named GARan to generate test cases [16]. In 2010, constraint-solving testing [12] was proposed as another way to obtain the test oracle in DBMS fuzzing, which provides the ground truth for the test cases. In 2015, SQLsmith [4] was released, which still generates statements based on predefined rules and can also connect to reference databases to serve as test oracles. Thanks to the open-source factor, compared to the SQL Server fuzzing tool, SQLsmith has rapidly attracted attention from both academia and industry. In 2020, Squirrel [116] first introduced execution feedback to improve code coverage in crash detection fuzzing. In the same year, Rigger et al. proposed several new fuzzing methods, including NoREC [90], TLP [91], and PQS [92]. NoREC and TLP apply metamorphic testing to the database domain, enabling fuzzing on a single DBMS. PQS is a constraint-solving testing method, and its oracle only requires that the execution result contain one row of the solving outcome, thereby accelerating the solving process compared to the former approach, which requires matching all the rows. In 2022, Liang et al. introduced SQLRight [68], a metamorphic testing method combined with execution feedback, and it is scalable to allow new oracles. In 2023, Tang et al. presented a constraint-solving testing method, TQS [101], which incorporated subgraph isomorphism search into DBMS fuzzing, significantly enhancing bug detection efficiency. In 2024, Ba et al. and Liang et al. respectively proposed DQP [15] and Mozi [67], further extending metamorphic testing to different execution paths of the same query.

DBMS fuzzing techniques can be classified from different perspectives.

- One classification approach is based on the way expected results are obtained, as shown in Figure 1: differential testing, metamorphic testing, and constraint-solving. Differential testing

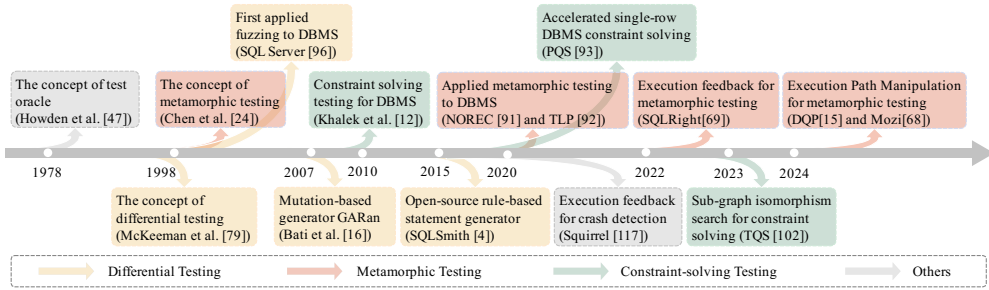


Fig. 1. Main milestones of DBMS fuzzing.

compares output differences across databases [77], mutation testing modifies input samples to assess the tolerance of the DBMS to exceptional inputs [24, 26, 62], and constraint-solving utilizes solvers for benchmark ground truths [21, 58, 79, 94].

- Fuzzing techniques can also be categorized by the transparency of the DBMS at test time: black-box, gray-box, and white-box [19]. Black-box testing assesses the behavior and functionality of DBMS solely through input and output observations [83]. Gray-box testing collects state information to generate targeted test cases [22, 23, 103], while white-box testing involves a deep understanding of the internal structure and emphasizes code coverage and quality assessment [38].

Existing fuzzing surveys focus mainly on *general-purpose* software. However, as the structures and testing objectives for different types of software can vary significantly, specific fuzzing methods [19] have to be carefully designed in order to tailor for each particular type of software system, e.g., DBMS. Compared to other types of software systems, DBMSs contain some unique modules and functionalities, including SQL optimizer, executor, transaction engine, and so on [41, 61, 65]. This leads to significant differences between DBMS and other software, e.g., how to generate test cases and how to verify the correctness of execution results. However, there does NOT exist any systematic survey that provides an in-depth study and comparison of DBMS fuzzing approaches. To fill this gap, in this article, we thoroughly review existing work on fuzzing for DBMSs, covering basic definitions, horizontal and vertical taxonomies, experimental comparisons, and future directions.

Specifically, the basic concepts and notations of DBMS fuzzing are introduced first, laying the foundation for further exploration. Then, we systematically define a general fuzzing procedure, we also decompose and categorize existing methods. The advantages, disadvantages, and contributions of the methods in each stage are also analyzed. In addition, we further classify existing methods from the testing objective perspective, covering various components in DBMSs. Recent studies have focused on fuzzing techniques for non-relational DBMSs; we briefly review these methods along with emerging techniques. In order to objectively evaluate the performance of each method, we present a comprehensive open-source toolkit, OpenDBFuzz. Based on this toolkit, we systematically test and analyze each method on SQLite, MySQL, TiDB, CockroachDB, and PostgreSQL with respect to bug type coverage, syntactic validity, semantic validity, code coverage, and the number of unique bugs detected. Finally, future research directions for DBMS fuzzing are discussed, identifying areas for further exploration and development. The contributions of this article are as follows.

- (1) We systematically define the general DBMS fuzzing procedure, decompose, and categorize existing methods. Furthermore, the advantages, disadvantages, and contributions of the methods in each stage are concluded.
- (2) We classify existing DBMS fuzzing methods based on testing components, encompassing various components in DBMSs, and provide detailed descriptions of representative works.

Table 1. Summary of Commonly Used Notations in This Article

Notation	Definition
Q	A SQL query, specifically refers to a SELECT statement
D	The target DBMS
S	A statement, which can be any SQL sentence supported by the target DBMS
p	A predicate in the statement
TX	A transaction, which begins with 'BEGIN' and ends with 'COMMIT' and contains some statements
TC	A test case, which contains some statements S s or some transactions TX s
t	A table in the target DBMS
$D(TC)$	The execution result of the test case TC in the DBMS D
$G(TC, D)$	The ground truth of the execution result for the test case TC in the DBMS D

- (3) We present OpenDBFuzz, the most comprehensive open-source toolkit for DBMS fuzzing, and conduct fair experimental comparisons to evaluate state-of-the-art methods using the same experimental configurations.
- (4) We thoroughly explore the issues and challenges of DBMS fuzzing and forecast future research directions in this field.

The remainder of this article is organized as follows: in Section 2, we introduce the preliminaries. Considering that the DBMS fuzzing process is complex and that the best method may not perform optimally at every step, we comprehensively disassemble, classify, and compare the existing methods from the horizontal perspective in Section 3. In addition, some fuzzing methods only target internal components in the DBMS. Therefore, in Section 4, we classify and describe representative fuzzing works from a vertical perspective. Section 5 extends the discussion to other non-relational types of DBMSs and emerging techniques. In Section 6, we present OpenDBFuzz and conduct a comprehensive experimental evaluation. Section 7 concludes the article and summarizes challenges.

2 Preliminary

This section provides an introduction to the notations, definitions, and backgrounds necessary for describing DBMS fuzzing techniques.

2.1 Notations

The notations and their explanations used in this article are summarized in Table 1. These notations will be used throughout the following sections. It is worth noting that adding an 's' to any notation indicates multiple instances of that item. For example, 'Ss' denotes multiple statements.

2.2 Basic Definitions

Database bugs encompass three main types: crashes, logic bugs, and performance bugs [73, 81]. Crashes occur when program errors or memory leaks interrupt the database while queries are executed. Compared to other types of bugs, crashes are relatively easier to identify. Logic bugs occur when the execution results deviate from the expected ones or violate common sense in the DBMS. Such bugs are usually hidden and difficult to detect. Performance bugs manifest themselves as significant differences in the execution time of the same query in different versions of DBMS, or large performance gaps between a query and its equivalent [35, 37, 40, 43, 92]. However, it is important to note that not all of these differences are considered to be actual bugs. Because determining the optimal execution plan is inherently an NP-hard problem [48], some databases practically choose suboptimal plans to speed up the optimization process. We consider performance

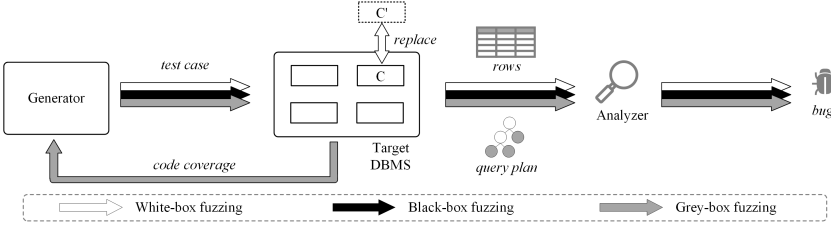


Fig. 2. Illustration of the differences between black-box fuzzing, grey-box fuzzing, and white-box fuzzing. The black arrow part represents the process of black-box fuzzing, which is also the basis of white-box and gray-box fuzzing. The gray and white arrows represent gray-box and white-box fuzzing, respectively.

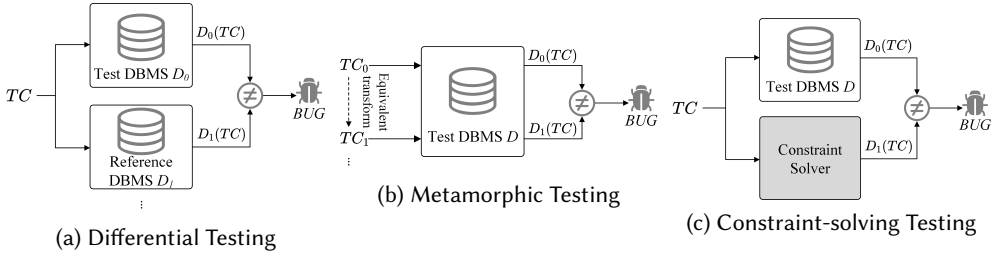


Fig. 3. Illustration of the differences between differential testing, metamorphic testing, and constraint-solving testing.

bugs to have less impact on normal database usage, and therefore, have a lower priority than the other two types of bugs.

DBMS fuzzing detects crashes, logic bugs, and performance bugs by generating a large number of SQL test statements in the DBMS [4, 12, 82, 88, 108]. According to the rules of statement generation, DBMS fuzzing methods can be divided into generation-based and mutation-based fuzzing. Generation-based fuzzing employs deterministic rules to randomly generate statements, while mutation-based fuzz testing expands statements by performing mutations on seed queries.

DBMS fuzzing can also be divided into black-box fuzzing, grey-box fuzzing, and white-box fuzzing according to the transparency of the target DBMS, as shown in Figure 2. Black-box fuzzing treats the DBMS as a black box, where a series of SQL statements or transactions are inputted, and a series of execution results or execution plans are obtained accordingly. The bugs are then determined by validating the results and plans. Grey-box fuzzing collects state information from the DBMS, such as code branch coverage and status, to guide statement generation and improve the efficiency of DBMS testing. By comparison, white-box fuzzing requires testing with an understanding of the internal structure and implementation mechanisms of the DBMS. For example, intramorphic testing [93] works by replacing a specific component in the system to get another version and then testing between the two versions to detect bugs in that component. However, white-box fuzzing is currently limited to theoretical research on general systems, and there is no white-box fuzzing tool for DBMS, which is also the focus of future research.

A testing oracle refers to a mechanism to obtain the expected results of test cases. DBMS fuzzing can also be divided into differential testing, metamorphic testing, and constraint-solving testing according to different oracles. Figure 3 shows the difference between them. Their formal definitions are shown in Definition 2.1, 2.2, and 2.3.

Definition 2.1 (Differential Testing). Differential testing detects bugs by comparing the results of the same test case between different DBMSs or versions. For a given test case TC , a target

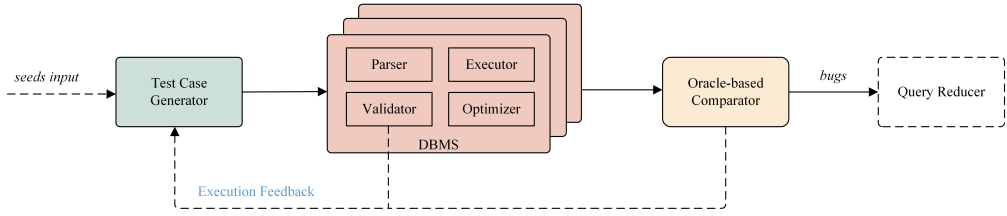


Fig. 4. General procedure of DBMS fuzzing. The green part represents the generator, the red part represents the DBMS, the yellow part signifies the oracle-based comparator, while the blue part corresponds feedback. These rules shall be applicable to this figure and the subsequent pipelines.

DBMS D_0 and some reference DBMSs $D_1, D_2, \dots, D_n$, the differential testing verifies that $\forall j : D_0(TC) = D_j(TC)$.

Definition 2.2 (Metamorphic Testing). Metamorphic testing applies predefined transformations to original test cases, in order to validate the expected relation between execution results before and after transformation. For a given test case TC_0 and a target DBMS D , some related test cases $TC_1, TC_2, \dots, TC_n$ can be obtained through transformation. Metamorphic testing verifies $\forall j : D(TC_0) \doteq D(TC_j)$, where $\doteq$ denotes a transformation-dependent relation. For equivalent transformations, $\doteq$ reduces to strict equality; for other transformation types, it represents the corresponding relation, such as set inclusion or inequality.

Definition 2.3 (Constraint-solving Testing). Constraint-solving testing utilizes constraint solvers to obtain the ground truth of query results and finds the DBMS bugs by comparing the ground truth with the eventual execution results. For a given test case TC , and a test DBMS D , constraint-solving testing first uses the constraint solver to obtain the ground truth $G(TC, D)$, and then verifies that $D(TC) = G(TC, D)$.

In the above definitions, we broadly define ‘=’ to indicate consistent executions. Any unexpected execution results, i.e., ‘ $\neq$ ’, are defined as bugs. This includes, but is not limited to:

- State Inconsistency: A process crash, assertion failure, a hang, or an internal error.
- Result Inconsistency: Data content, or row count.
- Transactional Inconsistency: Inconsistent visibility, Lock failure.

3 Fuzzing Step-based Taxonomy

The general (horizontal) procedure for DBMS fuzzing is illustrated in Figure 4. Basically, all fuzzers contain a test case generator and an oracle-based comparator. The former generates diverse test cases using different strategies to cover various functionalities of the DBMS as comprehensively as possible. The latter utilizes various types of oracles to analyze the execution results. If the execution results do not match the expected ones from the oracle, a bug is identified [104].

In addition to the aforementioned common modules, some fuzzers may also include the seed input, feedback, and query reducer modules. Seed inputs are typically combined with mutation-based approaches to enrich the state space of DBMS fuzzing. Feedback provides guidance for generating subsequent test cases based on execution information from the current test case [86]. The query reducer module is used to reduce the difficulty of triggering test cases to facilitate the localization and resolution of bugs by database developers [100].

3.1 Test Case Generator

The test case generator is the first step in DBMS fuzzing and is used to automatically generate test cases. There are primarily two types of implementation: generation-based and mutation-based.

Table 2. Comparison of Test Case Generators

Fuzzer	Year	Generator Type	Generator Strategy	Feedback	Database Instance	
RAGS[95]	1998	Generation-based	AST Model (Static Configuration)	No Feedback (Black-Box)	Existing Databases	
SQLsmith[4]	2015		AST Model (Dynamic Configuration)			
APOLLO[56]	2019					
AMOEBA[72]	2022		AST Model (Static Configuration)		Random Databases	
Go-Randgen[87]	2019					
SQLancer[92]	2020					
Artemis[80]	2021					
DT2[27]	2022					
DQE[96]	2023					
Troc[31]	2023					
Fucci[36]	2025					
TxCheck[54]	2023					
EET[55]	2024					
PUPPY[106]	2025					
DDLCheck[98]	2025					
TQS[101]	2023		AST Model (Dynamic Configuration)			
ADUSA[12]	2010		Alloy Model			
GARan[16]	2007	Mutation-based	SQL Structure Mutation	Feedback from internal status (Grey-Box)	Existing Databases	
Squirrel[116]	2020				Random Databases	
Squill[105]	2023					
SQLRight[68]	2022					
DynSQL[53]	2023					
Tuzz[64]	2024					
Eqsq[113]	2021			No Feedback (Black-Box)		
MutaSQL[25]	2020					
Kangaroo[63]	2023		SQL Sequence Mutation	Feedback from internal status (Grey-Box)		
SOFT[34]	2025					
LEGO[66]	2023			No Feedback (Black-Box)		
Griffin[33]	2022		DBMS State Mutation	Feedback from external interface (Black-Box)		
QPG[13]	2023					

Generation-based methods start from scratch to create test cases without any initial input. It typically relies on a specific SQL syntax model [102] to randomly generate constants and expressions, aiming to enumerate potential test cases that satisfy specific criteria or cover specific aspects of the DBMS. On the other hand, mutation-based methods begin with existing seed queries and apply various modifications or mutations to generate new test cases [94, 118]. In Table 2, we gain insight into the generation strategies of existing fuzzers and categorize them. Specifically, we compare them in terms of database instance, generation type, strategy, and feedback.

3.1.1 Database Instance. Before generating test cases, a database instance must be available. Depending on their sources, they fall into two categories: existing databases and random databases.

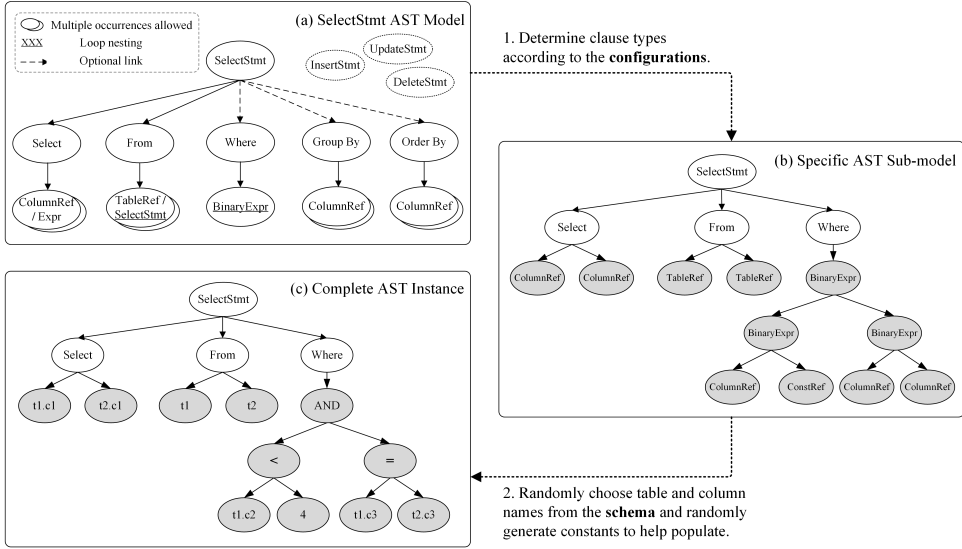


Fig. 5. Working process of generation-based generators based on AST model.

Existing databases come from benchmark tests such as TPC-H [1] or real datasets such as SCOTT [10]. Fuzzers based on existing databases [4, 16, 56, 72, 95] first acquire the schema of the target database and then generate syntactically correct statements based on that schema. However, as both the schema and the data are fixed, the generated statements cannot effectively explore the entire query space, leading to incomplete detection. On the other hand, fuzzers based on random databases [12, 14, 15, 27, 31, 33, 54, 80, 87, 91, 92, 96, 101, 105, 106, 116] create random database instances from scratch. The random generation of databases has been widely explored [20, 42, 45, 85]. A common practice is to first create tables, indexes, and views randomly and then populate them with data using INSERT, UPDATE, and DELETE statements [90]. The advantage of using randomly generated databases is their potential to thoroughly explore the query space. When coupled with execution feedback, these solutions can effectively explore the search space.

3.1.2 Generator Type and Strategy. After acquiring the database instance, we can proceed to generate test cases using either a generation-based or a mutation-based approach. Among them, the generation-based approach consists primarily of two strategies: **Abstract Syntax Tree (AST)** model and Alloy model [49].

The general process of the AST model is depicted in Figure 5. First, we need an AST model. AST itself is a tree-like data structure, which can represent the various components of SQL query statements (such as SELECT, FROM, WHERE, JOIN, etc.) and their relationships as nodes and branches for subsequent parsing and processing. Moreover, an AST model is a meta-model that covers the shared syntax structure of all SQL statements' AST instances. On top of that, the clause types are determined according to the configurations to obtain a specific AST sub-model. Then, table and column names are randomly selected from the schema of a specific database instance and combined with randomly generated constants to populate the AST sub-model, resulting in a complete AST instance. From the process, it can be observed that the AST model can only guarantee syntactic correctness, rather than semantic correctness.

In the AST model-based generator, the aforementioned configurations can be both static and dynamic. The static configurations refer to the static parameters manually set before generation, which can guide the generation of test cases. Common static configurations include:

- **Customizing the Probability of SQL Statements:** This parameter allows for customizing the distribution of test case types, such as the probability of SELECT, INSERT, UPDATE, DELETE, and so on [87, 95].
- **Customizing the Probability of SQL Clauses:** This parameter allows for customizing the distribution of different clauses, such as WHERE, GROUP BY, ORDER BY, and so on [87, 95].
- **Limiting the Maximum number of Joined Tables:** This parameter helps constrain the complexity of join relationships, preventing the generation of large-scale multi-table join statements and enhancing the efficiency of statement generation [87, 95].
- **Limiting the Maximum Number of Rows Returned:** This parameter restricts the execution time and resource consumption of the test cases by adding a LIMIT clause to the original statement [87, 95].
- **Limiting the Maximum Depth of the AST:** This parameter is set to limit the generation time of test cases, as the generation time tends to increase exponentially with the increase in AST depth. Without this limitation, numerous complex SQL statements would be generated, which may expand the search space but reduce the overall efficiency of bug detection [27, 31, 90–92, 96].

However, static configurations require manual adjustments based on specific requirements. This process can be time-consuming, and manually setting parameters may not sufficiently explore the state space in a targeted manner. Dynamic configurations of the AST model adjust generator parameters based on previous execution results, which include:

- **Dynamic Probability Table:** The dynamic probability table stores the dynamic probability of various SQL clauses. If an SQL query triggers a bug, the probability of the clauses appearing in that SQL query increases. This ensures that the generator explores the input space with maximum efficiency in generating test cases [56, 72].
- **Schema Graph:** The Schema Graph describes the connection relationships between tables, and by randomly traversing the schema graph, a join pattern can be obtained. It can adjust the selection probability of each table of the schema. This dynamic adaptation prevents the generation of duplicate test cases and significantly improves the efficiency of fuzzing [101].

Alloy [49] is a language for describing structured properties. When applied to describe SQL generation rules, it can specify not only SQL syntax rules, but also table names, column names, constant values in SQL statements, and constraints between table names and column names. As shown in Listing 1 and Listing 2, we provide a simplified example of using the Alloy model to generate SQL statements. The Alloy syntax model first defines four signatures: ‘FieldName’, ‘TableName’, ‘Value’, and ‘Table’. Then, ‘Term’ is defined to represent an item of Select clause, it consists of one FieldName and at most one AggregateName. The ‘Select’ signature contains multiple terms, while the ‘From’ signature contains multiple tables. Since the conditions in the ‘Where’ clause may be nested, ‘Condition’ and ‘ExtendCondition’ are defined to describe this possible relationship. The model defines a Condition using an Expr signature that represents a binary operation, where the FieldOrValue signature is used to constrain each operand to be, at most, one field or one value. Finally, by combining ‘Select’ clause, ‘From’ clause, and ‘Where’ clause, we construct a simple Select syntax model.

The Alloy data model describes the specific values for ‘FieldNames’ and ‘TableNames’. Additionally, it introduces the facts ‘select_field_in_table’ and ‘where_field_in_table’ place constraints on the column names appearing in the select and where clauses, requiring them to originate from tables stipulated in the from clause. The fact ‘unique_select_terms’ guarantees that there are no duplicate fields in the select clause. The Alloy syntax model is similar to the AST model, while the Alloy model guarantees the semantic accuracy of generated statements by constraining column

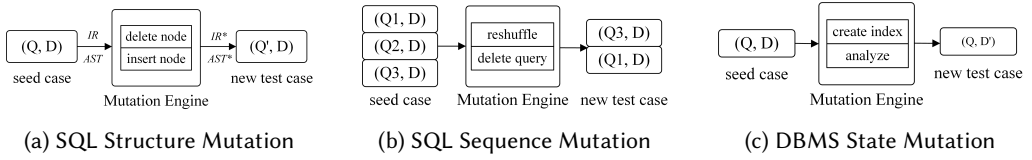


Fig. 6. Working process of mutation-based generators.

```

abstract sig FieldName {}
abstract sig TableName {}
abstract sig Value {}
abstract sig Table {
  name : one TableName,
  fields : some FieldName
}
sig Term {
  field : one FieldName,
  agg : lone AggregateName
}
sig Select {
  fields : some Term
}
sig From {
  tables : some Table
}
sig Where {
  condition : one Condition
}
... (Condition and Expr signatures)
sig SelectStatement {
  selectClause : one Select,
  fromClause : one From,
  whereClause : one Where
}

```

Listing 1. Alloy Syntax Model

```

sig id, name, studentID,
courseID, grade extends
  FieldName {}

sig students, grades extends
  TableName {}

fact select_field_in_table {
  all f: Select.fields.field |
  some t: From.tables |
  f in t.fields
}

fact where_field_in_table {
  all f: Where(...).field |
  some t: From.tables |
  f in t.fields
}

fact unique_select_terms {
  all a, b : SELECT.fields |
  (
    a.field = b.field and
    a.agg = b.agg
  ) => a=b
}

```

Listing 2. Alloy Data Model

names in select clause and where clause. Therefore, a generator based on the Alloy model [12] can achieve a higher level of semantic correctness compared to the AST model.

Mutation-based generators can be divided into three major categories: SQL structure mutation [16, 25, 34, 53, 63, 64, 68, 105, 113, 116], SQL sequence mutation [33, 66], and DBMS state mutation [13]. Figure 6 illustrates the differences between three mutation strategies. SQL structure mutation generates new test cases by modifying the structure or expression of statement, and it is the most common type of mutation. SQL sequence mutation generates new test cases by reshuffling or deleting queries from the original test cases. Its advantage is that it does not rely on SQL syntax, which reduces the adaptation cost for new DBMSs. DBMS state mutation uses DDL operations to create indexes or modify table definitions to generate new database schemas. In addition, it combines old SQL statements with the newly generated schemas to form new test cases. Since the same statement usually has different execution plans under different database schemas, and these diverse execution plans trigger distinct code branches, state mutations contribute to an increased degree of testing for the DBMS source code.

One of the implementation methods for the SQL structure mutation, known as the AST mutation, suffers from various limitations. Due to strict type constraints and complex mutation operations, altering the AST corresponding to an SQL statement is as challenging as modifying the SQL statement itself [116]. Therefore, the IR mutation was proposed to simplify mutation operations. As shown in Figure 7, IR nodes are actually AST tree nodes with constant values removed. Each IR node has a specific type and can have a maximum of two child nodes. Each mutation operation only needs to deal with the two child nodes of an IR node. There are three mutation operations applied to IR nodes:

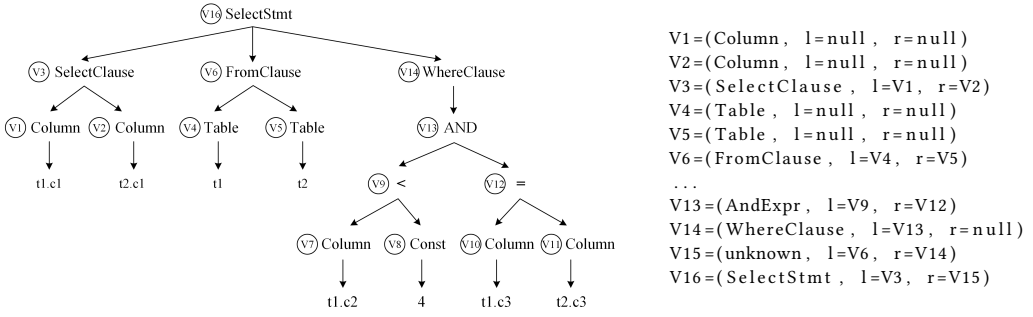


Fig. 7. An example of AST and its IR, on the left, the AST contains a series of tree nodes, and on the right, V1-V16 represent a series of IR nodes corresponding to each tree node. V15 does not correspond to any tree node in the AST, its introduction serves the purpose of conveniently representing the SelectStmt.

- **Insertion:** Insertion involves creating a new IR node and inserting it into an existing IR node. For example, a ‘Table’ can be inserted into a ‘FromClause’.
- **Replacement:** Replacement can be achieved by modifying the type of an IR node, such as replacing column ‘x’ with ‘count(x)’.
- **Deletion:** Deletion means removing at least one child node from an IR node.

3.1.3 Execution Feedback. Most generators do not take advantage of feedback information and treat the target DBMS as a black box, which accepts SQL statements or transactions as input and output execution results or execution plans. Some generators of black-box fuzzers [13] obtain feedback such as the validity of query plans or test cases from the query or query plan interface provided by the DBMS, guiding the subsequent generation process. This kind of feedback is essentially different from the feedback from gray-box fuzzers. The gray-box fuzzer generators [16, 53, 66, 68, 105, 116] collect the internal status of the DBMS, such as collecting code coverage to guide the generation of test cases that maximize code coverage. In contrast, the feedback of the black-box fuzzers relies only upon the external interface of the DBMS without any knowledge of the source code. Black-box fuzzers are easier to implement and can be used to test any commercial DBMSs. However, it is essentially a random exploration of the entire input space. Gray-box fuzzers guide generation based on the DBMS’s internal state and can usually achieve higher code coverage, but it can only be used for open source DBMSs. More types of feedback and information are elaborated on in Section 3.3.

3.2 Oracle-based Comparator

In manual testing, expected results can be calculated manually, and potential errors or vulnerabilities can be detected by comparing expected results with the eventual execution results on the target DBMS. However, for fuzzing, it is not a trivial task to obtain the expected results of random test cases. Oracle refers to the means to obtain the expected results [46]. Existing oracles can be classified into four types: crash oracle, differential oracle, metamorphic oracle, and constraint-solving oracle. Table 3 presents the classification of fuzzers along with their respective oracle implementation methods and applicable domains.

3.2.1 Crash Oracle. The crash oracle, also called a no-oracle, determines the occurrence of a crash based on the running state of the target database. Fuzzing methods based on crash oracle [33, 34, 53, 66, 105, 116] can only detect crashes. After executing test cases, bugs are determined based on whether the DBMS stops running.

Table 3. Comparison of Oracles

Fuzzer	Oracle Type	Feature	Test Scope
Squirrel[116]	Crash	Database Crashes	Crash Bugs
Squill[105]			
Griffin[33]			
LEGO[66]			
DynSQL[53]			
SOFT[34]			
RAGS[95]	Differential	Different DBMSs	Logic Bugs
SQLsmith[4]			
Go-Randgen[87]			
GARan[16]			
DT2[27]		Different Database Instances	
Radar[97]			
DDLCheck[98]			
APOLLO[56]		Different Versions of the Same DBMS	Performance Bugs
AMOEBA[72]	Metamorphic	Statement Rewriting	Logic Bugs
CERT[14]			
MutaSQL[25]			
Eqsq[113]			
NoREC[90]			
SQLRight[68]			
PINOLO[44]			
EET[55]			
CODDTest[112]			
PUPPY[106]			
Mozi[67]		Performance and Logic Bugs	
Kangaroo[63]		Logic and Crash Bugs	
DQP[15]		Statement Type Transformation	Logic Bugs
DQE[96]			
TLP[91]			
Troc[31]			
TxCheck[54]			
ADUSA[12]		Constraint-solving	
Artemis[80]	Backward Solving		
PQS[92]	Forward Solving(Logical Solver)		
TQS[101]			
WriteCheck[28]			
Fucci[36]			

3.2.2 Differential Oracle. Differential oracles detect logic bugs or performance bugs by comparing execution results between different DBMSs [4, 16, 27, 87, 95], different versions of the same DBMS [56], or the same DBMS operating under different constructions [97, 98]. It tries to get expected results from another database instance or DBMS. The advantage of differential oracle

<pre>CREATE TABLE t(x Integer, y Integer); + CREATE INDEX i ON t(x) WHERE p(x,y);</pre>	<pre>SELECT c1, c2 FROM t WHERE p; -- When q2 evals to empty SELECT c1, c2 FROM t LEFT JOIN q2 WHERE p;</pre>	<pre>SELECT c1, c2 FROM t WHERE p; -- When c1 does not contain null value SELECT c1, c2 FROM t WHERE p AND c1 IS NOT NULL;</pre>
(a) Constraint Rewriting	(b) Structure Rewriting	(c) Expression Rewriting

Fig. 8. Example of statement rewriting strategies.

testing is that it is relatively simple. Nevertheless, it cannot detect common bugs between the target DBMS and the referenced one.

3.2.3 Metamorphic Oracle. To avoid the drawbacks of the differential oracle, many fuzzing methods use the metamorphic oracle to detect logic bugs or performance bugs. Metamorphic oracle applies specific transformations on the original statements to construct new statements [24]. The execution results of the original and transformed statements are then checked to see if they satisfy a predefined metamorphic relation. Most relations are equivalence-based [15, 25, 54, 55, 68, 72, 90, 91, 96, 106, 112, 113], while only a small subset are non-equivalent [14, 44]. There are several ways to construct metamorphic oracles:

- **Statement Rewriting:** Statement Rewriting refers to the process of modifying a statement according to specified rules, so that the final semantics remain unchanged or change as expected. Common methods of rewriting statements include constraint rewriting, structure rewriting, and expression rewriting, as shown in Figure 8. Constraint rewriting refers to modifying the constraints of a table, such as creating an index. Obviously, the execution result of the same SELECT statement should remain unchanged after creating the index. Structure rewriting refers to adding new clauses such as JOIN, DISTINCT, and LIMIT based on the original statement. Expression Rewriting refers to the process of rewriting logical or arithmetic expressions, such as adding predicates that always hold true to the WHERE clause or changing the string comparison operator from '=' to 'LIKE', which will affect the result of the expression as expected [14, 25, 44, 72, 90, 112, 113].
- **Execution Path Manipulation:** Execution path manipulation is a technique that identifies bugs by manipulating the execution plan of a query and observing whether these manipulations produce divergent results or performance anomalies. For instance, query hints or system variables are employed to induce the same query to be executed using different plans, detecting performance or logic bugs [15, 67]. Similarly, a sequence of system variable operations can be executed prior to the query to trigger the optimizer to select varied execution plans, which is used to identify performance bugs [106]. In addition, some research [63] modifies the DBMS source code to force the same query to execute under multiple plans.
- **Statement Type Transformation:** Statement type transformation involves constructing UPDATE and DELETE statements that correspond to a given SELECT statement. For example, a row updated by an UPDATE statement with a predicate p should also be fetched by a SELECT statement with the same predicate p [96]. By adding tagging columns 'rowid' and 'updated' to the table, the SELECT statements can return 'rowid' of the affected rows, as shown in Figure 9. For DELETE statements, the affected rows can be determined by calculating the difference between the set of 'rowid' before and after deletion. Similarly, for UPDATE statements, the affected rows can be obtained by querying the 'rowid' where the 'updated' column is marked as true.
- **Query Partition:** Query partition refers to dividing an original SQL query into multiple partitions, and the merged results of the partitioned queries should be consistent with the results of the original query [91]. For example, a query 'SELECT * FROM t1' can be

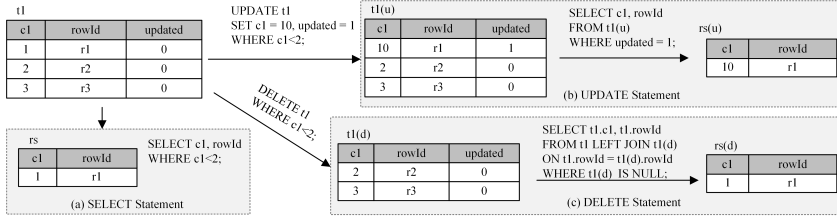


Fig. 9. Example of transformation of statement type.

partitioned into three queries: ‘SELECT * FROM t1 WHERE c1<10’, ‘SELECT * FROM t1 WHERE c1>=10’, and ‘SELECT * FROM t1 WHERE c1 IS NULL’;

- **Transaction Splitting:** Transaction splitting refers to splitting transactions into statements and then, by building a multiple version chain of data outside the database [31] or reorganizing the statement execution order [54], allowing statements within the transaction to run in non-transaction mode. By comparing the execution results before and after transaction splitting, it is possible to determine whether there exist any logic bugs in the transaction management module of the DBMS.

3.2.4 Constraint-solving Oracle. Due to the fact that not all test queries can obtain test oracles by constructing equivalent queries, some fuzzers use constraint-solving to obtain the expected result. Constraint-solving fuzzing methods [12, 80, 92, 101] generally rely on forward or backward solving to derive the ground truth of the execution results.

- **Forward Solving:** Forward solving employs an external solver to evaluate each tuple against predicate constraints in order to obtain the ground truth for the results of the statement execution. While traditional approaches utilize generic **SAT solvers** for this task solver [39, 59], recent methods, such as TQS [101], also use a logical external solver as a substitute for the SAT solver to derive the ground truth results based on the original test cases through logical operations or theoretical deduction [28, 36, 101].
- **Backward Solving:** Backward solving entails initially selecting some tuples as ground truth at random and then using an SAT solver to work backward and obtain a statement whose execution results include these tuples [92].

In all, each oracle has its own applicable scenarios. Fuzzing based on crash oracles can only detect crash-related issues such as program errors and memory leaks, which limits their application scenarios. Fuzzing based on differential oracles can only detect common functional modules between different databases and cannot identify logic or performance bugs shared by the referenced database and the test one. Fuzzing based on metamorphic oracles does not require a reference database, but suffers from a limited test range. Although constrained solvers are not subject to testing scope limitations, they exhibit lower solving efficiency, resulting in lower testing efficiency. Although the AST interpreter and logical operation solver improve the efficiency of constraint solving, they also suffer from the limitation of the test scope. For instance, the logical operation solver can only be applied to test logic bugs in join optimizations.

3.3 Execution Feedback

Execution feedback refers to the process of guiding test case generation based on execution results or database status. Execution feedback is not limited to the dynamic configuration of generators, but can also adjust mutation strategies and search directions, typically improving the efficiency of fuzz testing. Table 4 summarizes the feedback mechanisms used by existing fuzzers, which usually

Table 4. Comparison of Fuzzer Execution Feedbacks

Fuzzer	Metamorphic	Validation	Coverage	Query Plan	Syntax and Semantics Error
APOLLO[56]		✓			
Squirrel[116]			✓		
LEGO[66]			✓		
SQLRight[68]			✓		
QPG[13]				✓	
AMOEBA[72]	✓	✓			
GARan[16]			✓		
DynSQL[53]			✓		✓
Squill[105]		✓	✓		✓

include seven types: metamorphic feedback, validation feedback, coverage feedback, query plan feedback, syntax error feedback, semantics error feedback, and fitness feedback.

3.3.1 Metamorphic Feedback. Metamorphic feedback [72] refers to the use of feedback information to generate more statements that satisfy the conditions for the use of metamorphic oracles. This feedback is often combined with generation-based generators, where the generation process is influenced by the success of equivalent transformations. Specifically, in fuzzers using metamorphic feedback, statements are generated based on a probability table. If a SQL query can employ some of the statement rewriting strategies shown in Figure 8, the subsequent occurrence probabilities of LIMIT, GROUP BY, and WHERE clauses will increase. Metamorphic feedback improves the applicability between generated statements and the metamorphic oracle.

3.3.2 Validation Feedback. Validation feedback refers to the process of increasing the probability of generating test cases that can trigger bugs based on the feedback from the comparator. For generation-based generators [56, 72], if an SQL query triggers a bug, the probability of subsequent occurrence of different SQL clauses within the statement will increase. For mutation-based generators [105], if a mutation statement derived from a seed statement triggers a bug, validation feedback will add the mutation statement to the seed queue for selection by the generator.

3.3.3 Coverage Feedback. Coverage feedback aims to generate statements that maximize the coverage of the DBMS code. This feedback is commonly used in conjunction with mutation-based generators [16, 53, 66, 68, 105, 116]. By modifying the binary code of the target DBMS during execution to inject additional code that tracks and records execution paths, these fuzzers can collect code coverage information before and after executing each test case. If a mutation query leads to an increase in code coverage, it will be added to the seed queue. Additionally, LEGO [66] analyzes the type sequences of statements that increase the coverage of the code, and then the mutation processes will generate test cases containing the target type sequences.

3.3.4 Query Plan Feedback. The goal of query plan feedback is for the generator to maximize the number of query plans. A query plan represents the execution path of an SQL query in a DBMS, with different query plans corresponding to distinct code branches. Query plan feedback [13] uses the emergence of new query plans to guide subsequent query generation. Specifically, it models the objective of maximizing query plan quantity as a **Multi-Armed Bandit (MAB)** problem [18]. The generation of new query plans corresponds to exploitation, while exploring new mutation operations corresponds to exploration. When considering both exploitation and exploration, more unique query plans can be generated, increasing the possibility of discovering logic bugs.

Table 5. Comparison of Fuzzer Query Reducers

Fuzzer	Reduce Expression	Delete Clause	Delete Subquery	Delete IR Node	Semantics Preservation
RAGS[95]	✓	✓			
SQLancer[92]	✓	✓			
MutaSQL[25]	✓	✓			
GARan[16]	✓	✓			
SQLRight[68]				✓	✓
APOLLO[56]	✓	✓	✓		✓
DynSQL[53]	✓	✓	✓		✓

3.3.5 Syntax Error and Semantics Error Feedback. Some generators [105] consider statements with semantic and syntactic errors to be valuable because they may activate certain unexplored SQL features. Therefore, these statements are fed back into the seed queue, but given lower priority, rather than being discarded entirely. Some other generators [53] advocate that although syntax errors and semantic errors can improve code coverage, they do not help detect logic bugs. This is because, even though statements with syntax and semantic errors can trigger boundary branches in the code, they will fail during the parsing stage. This failure prevents them from exploring the underlying logic of the DBMS, providing limited help in detecting bugs. Therefore, feedback is utilized to filter out these invalid test cases from the seed queue.

Most fuzzers use coverage feedback to guide the generation of test cases. Intuitively, coverage feedback can help generate statements that cover as many code branches as possible, thus discovering various bugs. Validation feedback captures the characteristics of test cases that trigger bugs, effectively uncovering specific bugs, but it may not increase the variety of bugs discovered. Query plan feedback essentially generates statements with diverse query plans to trigger as many code branches as possible. Metamorphic feedback can guide the generation of test cases that are more suitable for oracles. Syntax and semantic error feedback play a minor role because test cases with syntax errors do not help detect the underlying bugs.

3.4 Query Reducer

Query reducers are designed to simplify and summarize complex queries, providing developers with minimal human-readable queries for debugging. Query reduction strategies can be classified into four types: reduce expression, delete clause, delete subquery, and delete IR node. Table 5 shows the query reduction strategies used by each fuzzer and also indicates whether the reducer preserves semantics.

Reducing expressions [16, 25, 92, 95] and deleting clauses refer to the process of reducing queries by simplifying WHERE clauses, arithmetic expressions, or logical expressions. They are relatively easy to implement, but cannot guarantee semantic correctness after simplification, resulting in a decrease in reduction efficiency. The removal of subquery operations [53, 56] extends the scope of simplification to subqueries, leading to a substantial improvement in the efficiency of simplification. Delete IR node operations are used in mutation-based fuzzers based on IR [68] mutation. They treat query simplification as a mutation operation on SQL queries, ensuring semantic correctness by deleting specific IR nodes. When considering the dependency between deleted expressions and remaining expressions, some fuzzers [53, 56, 68] have achieved semantic preservation.

4 DBMS Component-based Taxonomy

In this section, we review existing representative fuzzers from a DBMS component (vertical) perspective. As illustrated in Figure 10, existing DBMS fuzzers can be divided into overall fuzzing

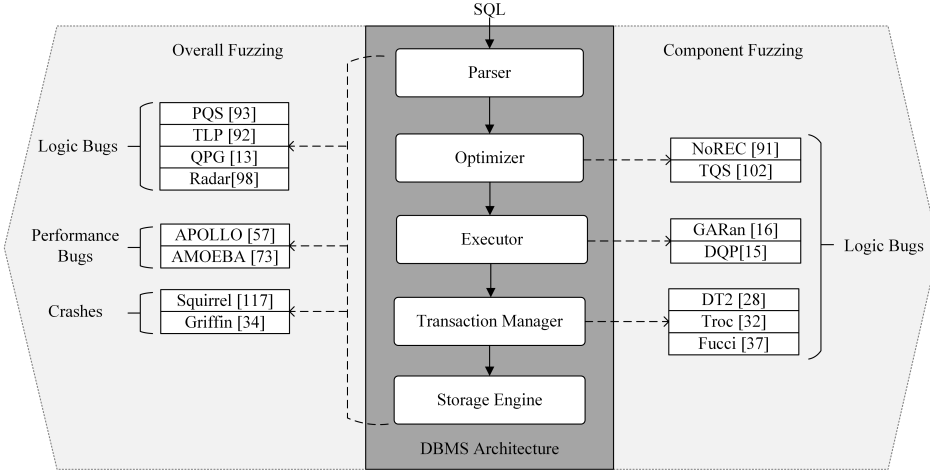


Fig. 10. Classification by DBMS components.

and component fuzzing. Overall fuzzing focuses on the entire DBMS, capable of detecting logic bugs, performance issues, or system crashes. Component fuzzing focuses on testing logic bugs in specific components of the DBMS, such as the optimizer, executor, and transaction manager.

4.1 Overall Fuzzing

Overall fuzzing is necessary for the initial development of DBMS, as it can comprehensively detect logic bugs, performance bugs, and crashes in the entire system. In terms of logic bugs, overall testing not only detects bugs within individual components of the DBMS but also identifies bugs that occur during the interaction between different components. For performance bugs, conducting component fuzzing alone on a single component is often insufficient for detection, as performance disparities typically result from the cumulative impact of multiple components. For crashes, fuzzers basically run against the entire DBMS. While it is possible to perform performance bug and crash detection targeting specific components, such testing is not only time-consuming but also provides only localized results, failing to capture the overall state of the entire DBMS. Therefore, there is little research effort in this area.

4.1.1 Overall Fuzzing on Logic Bugs. PQS [92] begins by randomly generating a pivot row as the ground truth and then constructs an SQL query that includes this pivot row in its result. By checking whether the result of the executed query in the test database contains the pivot row, logic bugs can be detected. Figure 11 illustrates the entire PQS pipeline. The main challenge of PQS is how to generate an SQL query whose execution result contains a known pivot row. The solution to PQS is to first create predicates randomly and then use the AST interpreter to evaluate whether the pivot row satisfies the predicate conditions. The evaluation result can be True, False, or NULL. True indicates that the pivot row can be queried through the predicate, requiring no adjustment. False indicates that the pivot row cannot be queried, so adding NOT before the predicate rectifies it. NULL indicates that the result of the predicate query is unknown, so adding IS NULL after the predicate can help. By employing the above method, PQS ensures that any random predicate can produce a satisfactory SQL query after being queried.

The core idea of TLP [91] is that the result of the predicate evaluation always falls within the values of True, False, and NULL. Therefore, an original query Q can be decomposed into three partitioned queries: $Q'_p, Q'_{-p}, Q'_p \text{ IS NULL}$. The expected result of the original query should be equal

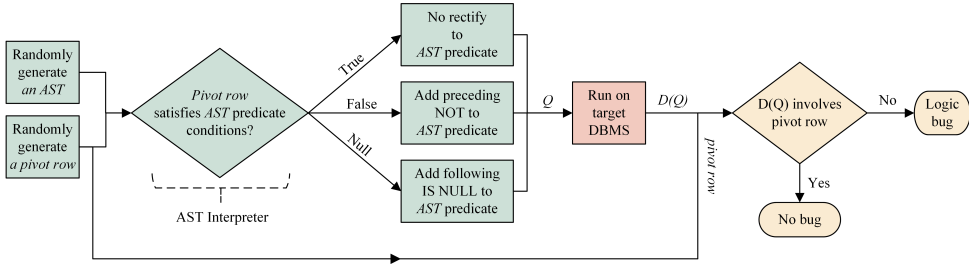


Fig. 11. Pipeline of PQS.

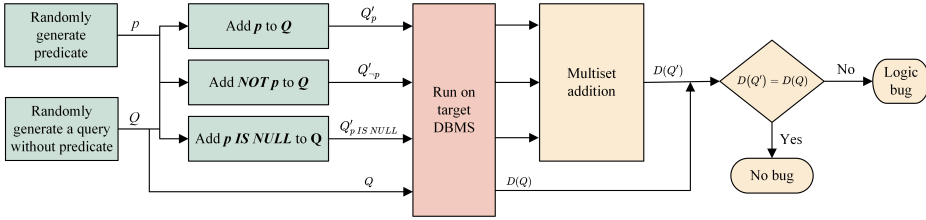


Fig. 12. Pipeline of TLP.

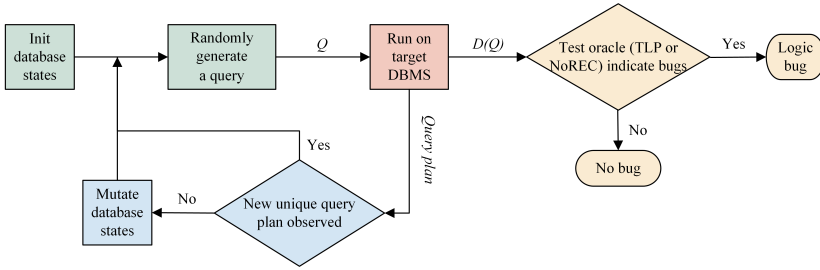


Fig. 13. Pipeline of QPG.

to the union of the results of the three partitioned queries. Figure 12 illustrates the main process of TLP. Using a generator similar to PQS, the original query is split into three equivalent partitioned queries, and metamorphic oracles are employed for result verification to detect logic bugs. Neither PQS nor TLP adopts feedback but performs a random search throughout the state space.

QPG [13] mutates the state of the database to generate more unique query plans, as shown in Figure 13. QPG implements a generation-based generator based on PQS, TLP, and NoREC. QPG also utilizes TLP and NoREC to validate query execution results. DDL and DML statements are used to alter the state of the database and obtain different query plans for the same query. Because different mutation operations contribute differently to generating new query plans, a decision among all mutation operators is necessary. The decision-making process for mutation operators is modeled as an MAB [18] problem. As shown in Equation (1), $\mu_i(t)$ represents the benefit of the i th mutation operator at time t , there is a probability of p for random selection, and a probability of $1 - p$ to opt for the mutation operator with the highest benefit. Since different query plans represent different execution paths, QPG can enhance the code coverage of DBMS.

$$operator(t) = \begin{cases} \operatorname{argmax}_{i=1 \dots k} (\mu_i(t)) & (1 - p) \\ \operatorname{random}(k) & (p) \end{cases} \quad (1)$$

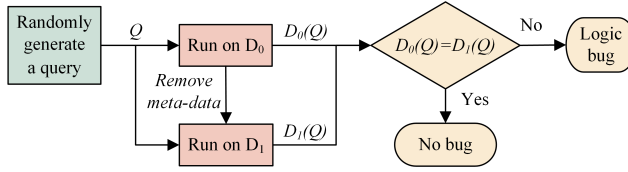


Fig. 14. Pipeline of Radar.

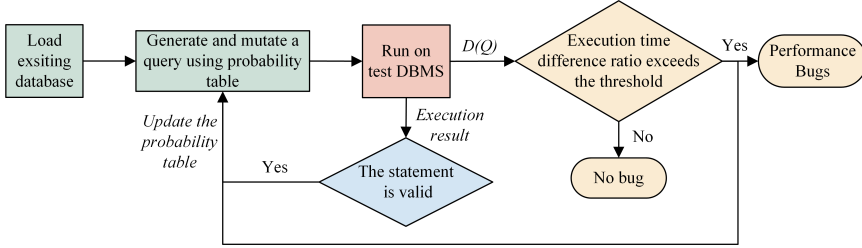


Fig. 15. Pipeline of AMOEBA.

Radar [97] constructs a database instance D_1 that contains the same data as a generated database D_0 in the same DBMS, and it checks for logic bugs by comparing the differences in SQL execution results between them. As shown in Figure 14, it first generates a database instance D_0 in the target DBMS by randomly constructing and executing statements to create a database schema and populate it with data. It then identifies metadata from this database, such as data types, column constraints, table constraints, indexes, and table configurations. Next, Radar constructs the schema for D_1 by removing all optional metadata, specifically indexes and secondary constraints, found in D_0 . The original data from D_0 is then copied into D_1 , creating two differential database instances. Finally, Radar uses schema-aware queries to query both databases and compares their execution results to detect logic bugs.

4.1.2 Overall Fuzzing on Performance Bugs. APOLLO [56] uses differential testing to detect performance regression bugs. It employs a generation-based generator with a dynamic probability table to generate many random queries. These queries are executed on different versions of the same database program, and performance bugs are detected using a differential oracle. APOLLO incorporates execution feedback to update the probability table, thereby improving the efficiency of the testing process. Additionally, Apollo uses query reduction that preserves semantic correctness to quickly obtain minimal test cases that developers can easily debug and fix.

AMOEBA [72] is based on metamorphic testing to detect performance bugs by comparing the execution time of equivalent queries. As shown in Figure 15, it utilizes a generation-based generator to generate a base query, on which it applies structure mutation and expression mutation operations to obtain mutant queries that are equivalent to the base query. Thanks to the feedback from the mutator and validator components, AMOEBA can effectively explore the query space and generate queries that can undergo equivalent mutations and are likely to trigger performance bugs.

4.1.3 Overall Fuzzing on Crashes. Squirrel [116] utilizes a mutation-based generator to mutate seed queries and detect crashes. It begins by randomly selecting a seed query from the seed queue and parsing it into an AST. The AST is then transformed into an IR form. SQL structure mutations are applied to the IR to generate new queries. By analyzing the logical dependencies between the arguments, the semantic validity of the query after mutation is ensured. Squirrel also utilizes coverage feedback to guide the generator towards exploring the query space with maximum code coverage.

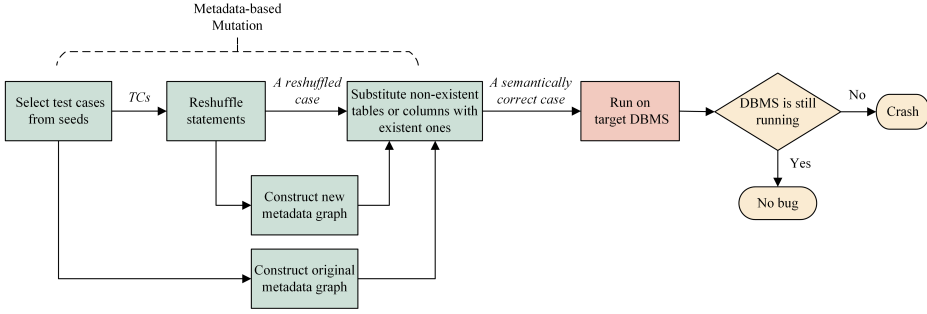


Fig. 16. Pipeline of Griffin.

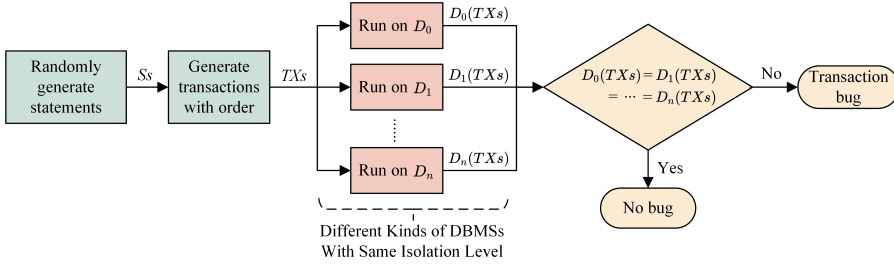


Fig. 17. Pipeline of DT2.

Griffin [33] utilizes SQL sequence mutation to generate a large number of test cases to detect crashes. As shown in Figure 16, Griffin first selects some test cases from the seed input and constructs a metadata graph for them, which describes the relationships between the statements and the tables or columns they create or access. A reshuffled case is then generated by deleting some statements and reordering the SQL sequence, but this may introduce semantic errors. By substituting non-existent tables or columns with existing ones, a semantically correct case can be obtained. Since mutations are performed at the statement sequence level, Griffin does not require dedicated parsers implemented for each DBMS, thus reducing adaptation costs.

Overall fuzzing tests the DBMS as a whole. The advantage is that the test coverage is wide and can detect potential bugs that may be difficult to notice. However, it has the disadvantages of low bug detection efficiency and difficulty in pinpointing the root cause of bugs. In some scenarios, it may be sufficient to just test a specific module to check whether its modifications introduce new bugs, while conducting overall testing may appear redundant and unnecessary.

4.2 Transaction Testing

Transactions are used to maintain the consistency and integrity of the data in the DBMS. Logic bugs in transaction implementation can lead to serious consequences, such as incorrect query results and unexpected DBMS blockage. Some testing tools [27, 31] have already confirmed the presence of transaction isolation vulnerabilities in widely used DBMSs, where they fail to achieve transaction isolation levels as claimed. Therefore, conducting fuzzy tests for transactions is crucial.

As illustrated in Figure 17, DT2 [27] randomly generates statements and combines a series of statements with BEGIN and COMMIT or ROLLBACK to form a transaction. DT2 then randomizes the commit order of statements in all transactions. Finally, transaction bugs are detected by comparing the execution results of transactions on different DBMSs.

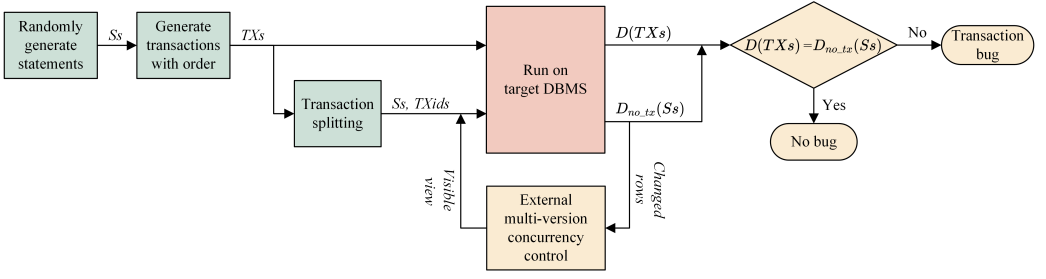


Fig. 18. Pipeline of Troc.

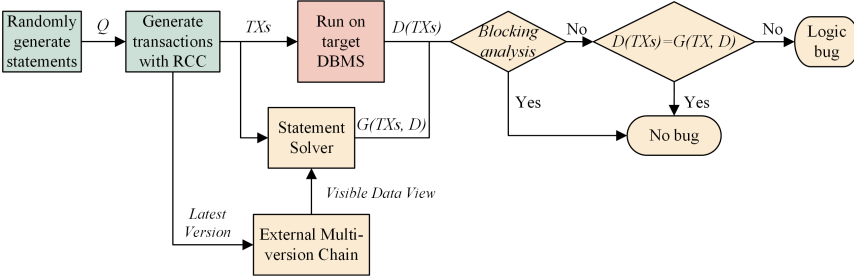


Fig. 19. Pipeline of Fucci.

Troc [31] utilizes metamorphic testing to detect logic bugs in transactions. The Troc pipeline is illustrated in Figure 18. Firstly, Troc generates random transactions and random submission orders. It is then tested in two equivalent scenarios: transaction and non-transaction modes. Transaction mode refers to submitting multiple transactions directly to the database. In non-transaction mode, transactions are split into separate statements with transaction ids. Troc constructs an external multi-version concurrency control system based on the changed rows in each execution. This ensures that every statement executed in non-transaction mode can obtain the same visible view as in transaction mode. Transaction bugs can be detected by comparing the execution results in transaction and non-transaction modes.

Fucci [36] utilizes a fully external constraint-solving oracle to verify the correctness of transaction execution. As shown in Figure 19, by leveraging an AST-based solver and a multi-version data chain, this oracle calculates precise expected results, including blocking conditions, query result sets, and the final state of the database. Furthermore, Fucci incorporates a **Random Conflict Construction (RCC)** mechanism, which classifies and models conflict combinations to strategically construct transaction pairs, thereby significantly increasing the probability of triggering transactional conflicts. Finally, the framework introduces an efficient test case reducer to streamline bug reporting. Compared to state-of-the-art approaches such as DT2 and Troc, Fucci demonstrates superior detection efficiency and covers a broader exploration space for identifying isolation anomalies.

4.3 Optimizer Testing

The optimizer is one of the most complex components of a DBMS. It analyzes different execution plans of a query and selects the best query plan by considering indexes, association conditions, data statistics, and other factors to minimize query time and resource consumption. However, implementation errors in the optimizer can lead to serious logic bugs.

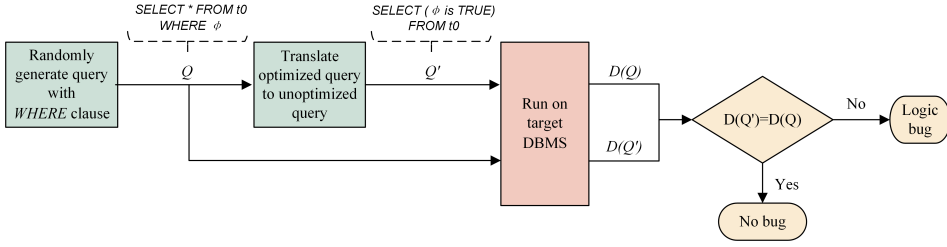


Fig. 20. Pipeline of NoREC.

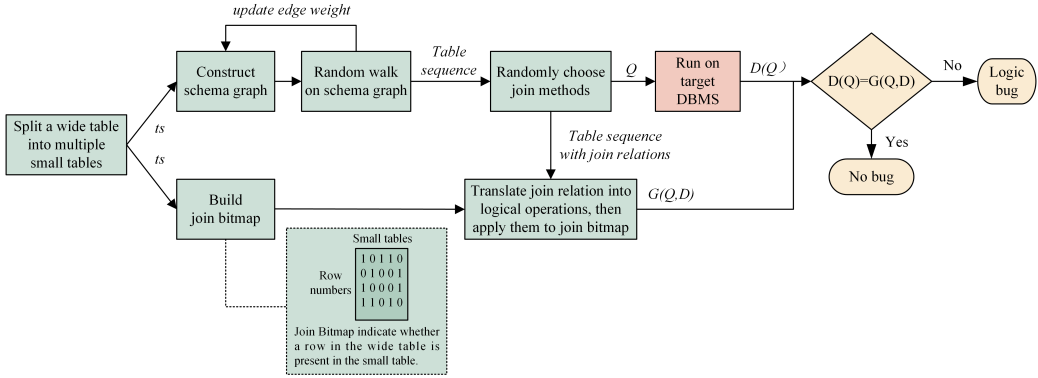


Fig. 21. Pipeline of TQS.

The NoREC pipeline [90] is illustrated in Figure 20. NoREC uses an AST model-based generator to generate queries with WHERE clauses and obtains the unoptimized query by moving the conditions from the WHERE clause to the SELECT clause. The presence of optimizer logic bugs is determined by comparing the number of rows returned by the original query and the number of ‘True’ returned by the unoptimized query.

The pipeline of TQS [101] is illustrated in Figure 21. Firstly, TQS splits a wide table into multiple small ones. On the one hand, it constructs a schema graph based on the foreign key relationship. By performing random walks on the schema graph, table sequences can be obtained. Combining these table sequences with randomly selected join methods, a multi-table SQL query ‘Q’ can be generated that does not include any filter predicate. On the other hand, it creates a join bitmap that indicates whether a row in the wide table exists in the small tables. The join relations of an SQL query can be translated into logical operations on the join bitmap. For example, ‘t1 inner join t2’ can be translated into the logical ‘AND’ operation between two columns of the join bitmap. By applying logical operations to the join bitmap, the ground truth for ‘Q’ can be derived. Comparing the actual execution results of ‘Q’ on the target DBMS with its ground truth enables the detection of any potential logic bugs. It is important to highlight that after each random walk, the edge weights of the schema graph are updated. This reduces the likelihood of generating duplicate queries and enhances the efficiency of detecting logic bugs.

4.4 Executor Testing

The executor is responsible for translating a query plan into a combination of physical operators and executing them to obtain the query result. These physical operators are specific instructions for performing query operations, such as Index Scan, Hash Join, Sort, and so on. During the

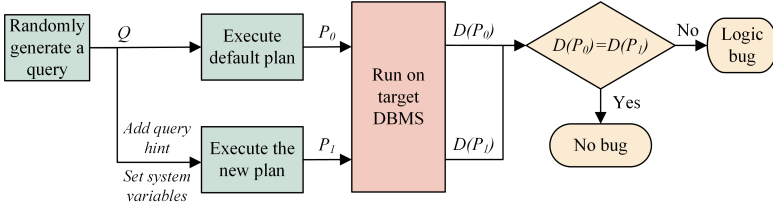


Fig. 22. Pipeline of DQP.

development process of DBMS, any modification of the operator must ensure its correctness in various combinations. Therefore, fuzzing needs to be performed under different query plans that contain various combinations of physical operators.

GARan [16] utilizes a mutation-based generator with genetic algorithms to focus on checking executor logic bugs. By setting a fitness function that evaluates the quality of mutated SQL statements, GARan can generate statements that produce specific physical operators, such as Hash Join. Then, based on the idea of differential testing, it compares the execution results of the same statement on different DBMSs.

DQP [15] detects logic bugs by enforcing the DBMS to utilize distinct execution plans, P_0 and P_1 , for the same query Q and identifying inconsistencies in their execution results. In Figure 22, $D(P_0)$ and $D(P_1)$ denote the execution outcomes of query Q on DBMS D utilizing the corresponding plans. The approach is generator-agnostic and influences plan selection via two primary mechanisms. The first mechanism involves injecting optimization-related hints directly into the generated query. These hints instruct the optimizer to alter specific components or strategies within the execution plan. The second mechanism alters system variables to modify optimization options, effectively enabling or disabling specific methods that the optimizer can use during plan generation. Finally, DQP compares execution results under these plan manipulations with those from the default execution to determine whether a logic bug exists.

5 Non-relational DBMS Fuzzing and New Techniques

Beyond fuzzing for relational DBMSs, recent work has expanded to non-relational DBMSs, including graph, vector, spatial, distributed, and GPU databases. In addition, some studies leverage **Large Language Models (LLMs)** to enhance fuzzing. This section summarizes these emerging efforts.

5.1 Graph Database Management System (GDBMS) Fuzzing

GDBMSs are non-relational systems for managing and querying highly interconnected data, typically represented using two paradigms: the Property Graph model and the **Resource Description Framework (RDF)** model. The widely adopted Property Graph model represents data as nodes and edges with attached properties, whereas the RDF model, a W3C standard, represents data as collections of subject–predicate–object triples. To query these structures, diverse languages have emerged, including declarative pattern-matching languages such as Cypher, SQL-like languages such as nGQL, and imperative graph traversal languages such as Gremlin. This diversity in data models, query semantics, and language paradigms poses challenges for ensuring the correctness and robustness of graph database systems. Table 6 summarizes existing GDBMS fuzzers and their implementation characteristics.

As pioneering fuzzers for GDBMSs, Grand [115] and GDSmith [70] established representative pipelines for database instance and query generation, which most subsequent fuzzers have built upon. For Gremlin, query generator models callable APIs, defining rules for random traversal, and synthesizing the final query by constructing valid method invocation chains. In contrast, Cypher

Table 6. Comparison of GDBMS Fuzzing

Fuzzer	Year	GDBMS Type	Language	Oracle	Feature
Grand[115]	2022	Property	Gremlin	Differential	Different GDBMSs
GDsmith[70]	2023		Cypher		
RD2[109]	2023	RDF	SPARQL		
GDBMeter[57]	2023	Property	Gremlin and Cypher	Metamorphic	Query Partition
Gamera[119]	2023				Statement Rewriting, Data Manipulation
GraphGenie[51]	2024				Statement Rewriting
GRev[76]	2024		Cypher, Gremlin, nGQL		
GraspDB[71]	2024		Cypher		
QuDi[114]	2024		Gremlin		Query Splitting
GQS[111]	2025		Cypher	Constraint-solving	Backward Solving
Gslicer[75]	2025			Differential	Different Graphs

queries are typically generated by first creating a query skeleton and then recursively populating it with patterns and expressions. RD2 [109] further provides an implementation for fuzzing the SPARQL query language under the RDF model.

More recent studies—including GDBMeter [57], Gamera [119], GraphGenie [51], GRev [76], GraspDB [71], QuDi [114], GQS [111], and Gslicer [75]—focus on in-depth exploration of test oracles. While these oracles share conceptual similarities with those used in relational DBMS fuzzing, they are specifically tailored to the unique characteristics and challenges of GDBMSs.

5.2 Fuzzing for Other DBMS Types

Unicorn [107] targets **Time-Series Database Management Systems (TSDBMSs)** using a hybrid generator that produces valid queries by mutating time-series-specific AST nodes. It injects an agent into each TSDB process to intercept runtime exceptions silently suppressed by built-in exception handling and report them to an analyzer, thereby exposing otherwise hidden crashes.

Spatter [30] focuses on **Spatial Database Management Systems (SDBMSs)** and employs affine equivalent inputs as its test oracle. It first generates spatial instance from random geometric shape and applies predefined derivative strategies to create topologically related geometries. By further constructing affine-transformed database instances (e.g., via rotation, scaling, or translation) and comparing query results across equivalent instances, Spatter detects spatial logic bugs.

SQLaw [47] is a fuzzer for GPU DBMSs that detects bugs by comparing the execution results of the same query on GPU and CPU backends. To address the limited coverage of GPU execution paths, SQLaw adopts a mining-based approach. It generates raw data and queries, labels each query by its execution backend, and applies association rule mining to extract frequent system-configuration and query-structure patterns. By analyzing their confidence with respect to execution paths, SQLaw identifies “CPU-only” and “GPU-only” query patterns and further enriches test generation through expression mutations, increasing the likelihood of exercising GPU-specific logic.

DepState [32] is a fine-grained fuzzer for detecting synchronization bugs in distributed databases (DDBMSs). It constructs sharded database instances with explicit cross-partition dependencies and focuses on generating operation sequences that directly affect cluster state. By monitoring synchronization-related code coverage, DepState identifies precise timing windows during which cluster state changes occur and strategically injects cluster-modifying operations within these windows. Coverage feedback from these executions is then used to guide the exploration of a broader range of state-affecting operation sequences.

Additionally, SQLxDiff [52] enables differential testing for emerging systems—including TS-DBMSs, DDBMSs, and streaming DBMSs—by mapping their queries to equivalent constructs in PostgreSQL. BUZZBEE [110] extends fuzzing to non-relational systems such as key-value, graph, and document DBMSs. Inspired by Intermediate Representation techniques, BUZZBEE adapts ASTs to uniformly model heterogeneous syntaxes and enforces semantic validity through annotation-guided constraints resolved by a custom parser. Coni [29] further broadens fuzzing to database connectors by modeling connector functions as state machines and performing differential testing across different connectors using generated operation sequences.

5.3 LLM in Fuzzing

Recent studies integrate LLMs into DBMS fuzzing generators and oracles. LLMs have been explored for tasks such as database dialect translation, schema-aware statement mutation, surrogate oracle construction based on learned bug patterns, and execution-log analysis for latent bug detection [50]. For instance, QTRAN [69] proposes a two-phase framework to improve cross-dialect test portability, combining RAG-enhanced SQL translation with a fine-tuned LLM that performs dialect-aware mutation in the target environment. Similarly, ShQveL [117] adopts an LLM-guided approach for multi-dialect testing by decoupling structural consistency from dialect-specific features, using SQL sketches and LLM-based fragment synthesis to inject complex dialect constructs. Additionally, a **Reinforcement Learning (RL)**-guided LLM framework [74] iteratively optimizes prompt templates for test generation based on oracle feedback from discovered bugs or coverage improvements.

6 Benchmarks and Comparison

As can be seen from the previous sections, there are quite a number of DBMS fuzzers and their structures are quite complex. In order to objectively evaluate the performance of each fuzzer, we present a comprehensive open-source toolkit, OpenDBFuzz,¹ which integrates five relational DBMS (SQLite, MySQL, TiDB, CockroachDB, and PostgreSQL) and eight fuzzing tools (SQLancer, DQE, AMOEBA, SQLRight, Squirrel, Troc, APOLLO, and Radar), and provides entry function, interactive interface, runtime configuration files, and detailed command prompts. Based on this toolkit, we conduct fair experimental comparisons using the same configurations and focus on answering the following questions:

- Q1:** (Between fuzzers) For each bug type, which fuzzer performs best under a unified testing environment?
- Q2:** (Fuzzers themselves) Which bug types can each fuzzer detect, and do they exhibit the advantages they claim?
- Q3:** (Inside fuzzers) Which module approach performs best among the various steps of fuzzing?

Therefore, in this section, we first introduce the database instances and the open-source fuzzers, followed by a classification of the bugs they discover. We then define the evaluation metrics and systematically evaluate existing fuzzers with respect to logic bugs, crashes, and performance bugs.

6.1 Database Instances and Test Cases

Some generation-based fuzzers start from scratch to generate database instances, while others, such as SQLsmith [4], require existing database instances. For the latter experiments, we use TPC-H as the database instance. The mutation-based fuzzers generate diverse test cases by mutating seed statements. We use official unit test cases provided by DBMS vendors as our seed corpus, including SQLite TCL test scripts [99], PostgreSQL official test infrastructure [89], and MySQL unit test samples [84].

¹<https://github.com/Reverie4u/OpenDBFuzz>

Table 7. Source Code of Related Fuzzers

Fuzzer	Supported DBMS	Link
SQLancer[92] (PQS[92], NoREC[90], TLP[91], QPG[13], CERT[14], DQP[15], CDDTest[112])	SQLite, MySQL, TiDB, MariaDB CockroachDB, DuckDB, OceanBase	https://github.com/sqlancer/sqlancer
DQE[96]	SQLite, MySQL, MariaDB TiDB, CockroachDB	https://github.com/tcse-iscas/dqetool
SQLRight[68]	SQLite, PostgreSQL, MySQL	https://github.com/psu-security-universe/sqlright
SQLsmith[4]	SQLite, MonetDB, PostgreSQL	https://github.com/anse1/sqlsmith
Go-Randgen[87]	MySQL, TiDB	https://github.com/pingcap/go-randgen
Squirrel[116]	SQLite, PostgreSQL, MySQL, MariaDB	https://github.com/s3team/Squirrel
DT2[27]	MySQL, MariaDB, TiDB	https://github.com/tcse-iscas/DT2
Troc[31]	MySQL, MariaDB, TiDB	https://github.com/tcse-iscas/Troc
APOLLO[56]	SQLite, PostgreSQL	https://github.com/sslab-gatech/apollo
AMOEBa[72]	PostgreSQL, CockroachDB	https://bit.ly/3I995jL
TxCheck[54]	TiDB, MySQL, MariaDB	https://github.com/JZuming/TxCheck
WriteCheck[28]	MySQL, PostgreSQL, SQLite MariaDB, CockroachDB, TiDB	https://github.com/tcse-iscas/WriteCheck
PINOLO[44]	MySQL, MariaDB, TiDB, OceanBase	https://github.com/qaqcatz/impomysql-git
EET[55]	MySQL, PostgreSQL, SQLite ClickHouse, TiDB	https://github.com/JZuming/EET
Radar[97]	SQLite, MySQL, MariaDB CockroachDB, TiDB	https://github.com/tcse-iscas/radar
DDLCheck[98]	SQLite, MySQL, TiDB PostgreSQL, MariaDB, CockroachDB	https://figshare.com/s/e222bd56c4925375ac34
Fucci[36]	MySQL, TiDB, MariaDB	https://github.com/Reverie4u/Fucci

6.2 Open-source Fuzzers

Source code is crucial for researchers to reproduce and compare the fuzzing methods. We have summarized the code of all open-source fuzzers mentioned in Section 3 and the DBMSs they support in Table 7. The following comparative experiments were conducted on representative fuzzers among them.

6.3 Detection Capabilities across Different Bug Types

To systematically characterize the bug detection capabilities of different fuzzing methods and analyze the intersection among their findings, we construct a bug taxonomy comprising four top-level categories. Based on this taxonomy, we present statistical summaries in Table 8 (a)–(d), which highlight differences in bug type coverage across various fuzzing methods.

The results indicate that, for logic bugs, DQP and PQS cover all six subtypes; for crash bugs, SOFT, SQLsmith, and TxCheck provide the most comprehensive coverage; for performance bugs, APOLLO achieves the broadest detection scope; and for transaction bugs, no existing method covers all subtypes, with DT2 and WriteCheck detecting the largest number (three subtypes each). It is worth noting that limited coverage by other methods does not necessarily imply inferior performance, but often reflects deliberate design choices and targeted objectives. The complete classification criteria and the sources of the bug list are detailed in the Appendix A, and the full list is released as part of OpenDBFuzz.

6.4 Evaluation Metrics

We adopt the standard metrics for evaluating fuzzing performance:

Table 8. Overview of Fuzzer Detection Capabilities Across Different Bug Types

(a) Logic Bugs Detection							(b) Crash Bugs Detection						
Fuzzer	Interface & Pars.	Type & Seman.	Query Opt.	Exec. Engine	Feature Inter.	State & Resource	Fuzzer	Buffer Overflow	Memory Lifecycle	Illegal Memory	Resource Exh.	Internal Logic	Arith. Errors
MutaSQL			✓	✓	✓		squirrel	✓	✓	✓	✓	✓	
Esq1	✓	✓	✓	✓		✓	Squill	✓	✓	✓		✓	
ADUSA			✓				Griffin	✓	✓	✓		✓	
NoREC	✓	✓	✓	✓	✓		LEGO	✓	✓	✓		✓	
SQLRight		✓	✓	✓	✓	✓	DynSQL	✓	✓	✓		✓	✓
DQE	✓	✓	✓	✓			SOFT	✓	✓	✓	✓	✓	✓
TLP	✓	✓	✓	✓	✓		SQLRight	✓		✓		✓	
TQS	✓	✓	✓	✓		✓	SQLsmith	✓	✓	✓	✓	✓	✓
PINOLO		✓	✓	✓	✓		TLP		✓				
EET		✓	✓	✓	✓	✓	NoREC					✓	
DQP	✓	✓	✓	✓	✓	✓	DQP					✓	✓
CODD		✓	✓				CODD						✓
Radar		✓	✓	✓	✓	✓	DDLcheck	✓		✓	✓	✓	✓
DDLcheck		✓	✓			✓	Kangaroo			✓	✓	✓	
Kangaroo	✓		✓		✓		PQS			✓	✓		
PQS	✓	✓	✓	✓	✓	✓	Radar	✓					✓
Mozi	✓	✓					EET	✓		✓	✓	✓	
SQLsmith	✓	✓		✓	✓	✓	DQE						✓
DT2		✓					TxCheck	✓	✓	✓	✓	✓	✓
Troc		✓					WriteCheck	✓					
Fucci		✓											
TxCheck			✓	✓									
WriteCheck			✓			✓							
(c) Performance Bugs Detection							(d) Transactional Bugs Detection						
Fuzzer	Query Rewriting	Plan Selection	Execution Loss				Fuzzer	Lock Management	MVCC Visibility	Invalid State Transition	Recovery Rollback		
AMOEBa	✓		✓				DT2	✓	✓	✓			
APOLLO	✓	✓	✓				TRoc	✓	✓				
CERT	✓		✓				TxCheck	✓	✓				
PUPPY	✓	✓	✓				WriteCheck	✓	✓		✓		
Mozi	✓	✓	✓				Fucci	✓	✓				
							Esq1				✓		
							PQS				✓		

- **Number of bugs:** The number of bugs can directly quantify the bug detection capabilities of different fuzzers. Some studies submit the detected bugs to the open-source database community to obtain the number of community confirmed bugs. However, to make comparative experiments more discriminative, we need to conduct subsequent experiments on older versions of the DBMSs, so this method cannot be used. To avoid overrating methods that output highly redundant bugs, we used expert filtering to remove the obviously repetitive bugs, thereby improving the rationality of this metric.
- **Validity:** Validity measures the proportion of test statements that are syntactically and semantically correct. Only valid statements can trigger the underlying DBMS logic and thus enable bug detection. In our experiments, validity is defined as the ratio of generated valid statements to all generated statements.
- **Valid Cases Per Second:** Valid cases per second are the number of valid test cases executed by the fuzzer within a given time range. It measures the overall efficiency of generation, execution, and comparison in a fuzzer.
- **New Edges:** New edges refer to the number of code branches explored during fuzzing, indicating the breadth of the testing scope.
- **Coverage:** Coverage indicates the proportion of code branches that have been explored. It is calculated using $new\ edges / total\ edges$, where $total\ edges$ represents the number of all code branches in the DBMS.

6.5 Logic Bugs Detection Comparison

Since the feedback module is relatively pluggable, we first tested the fuzzers without feedback to eliminate its impact on the evaluation of other modules and then tested the feedback module independently. For logic bug detection, we conducted comparative experiments on six fuzzers,

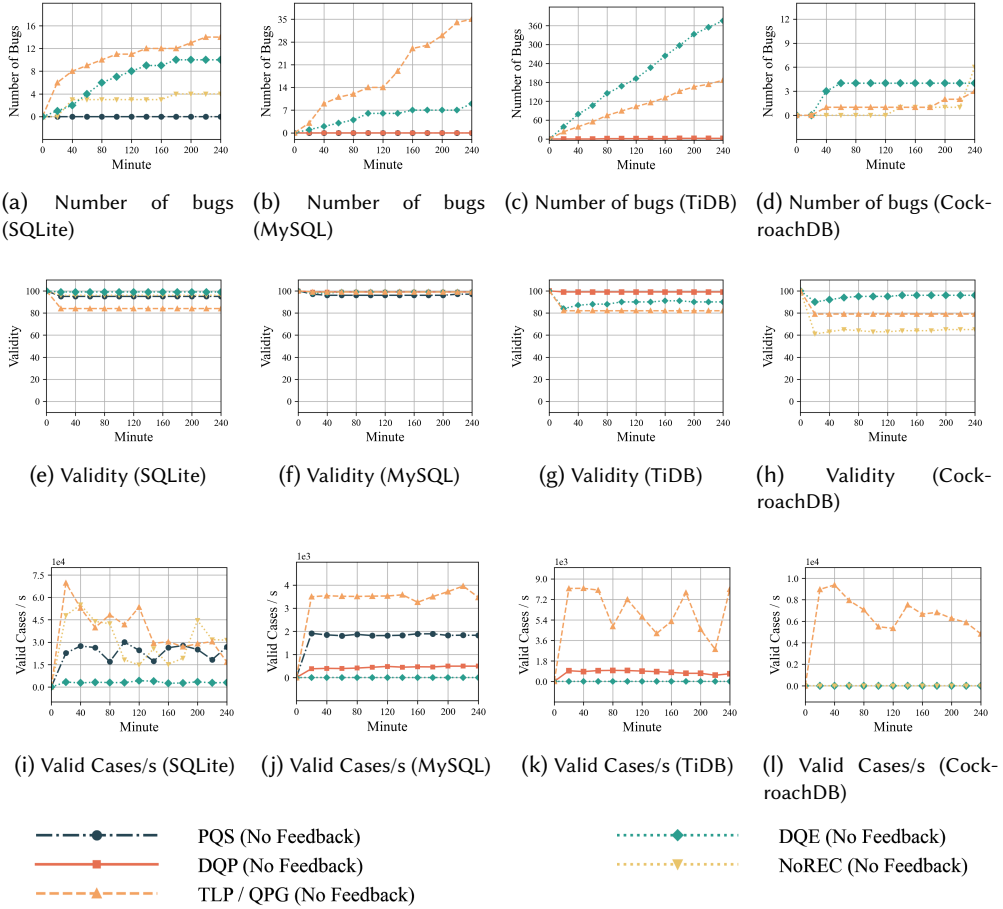


Fig. 23. Comparison of oracles for detecting logic bugs.

including PQS, NoREC, TLP, QPG, DQP, and DQE. Among them, TLP and QPG only differ in feedback, and the other modules are consistent. Experiments were conducted on four popular DBMSs: MySQL 8.0.39, SQLite 3.47.2, TiDB 8.4.0, and CockroachDB 25.1.0. The number of bugs, the validity, and the valid cases per second are used as evaluation metrics.

We conducted a 240-minute evaluation, with results shown in Figure 23. The results reveal substantial differences across methods and databases in terms of the number of bugs, validity ratios, and the throughput of valid cases per second, largely attributable to differences in statement generation strategies and oracle designs. TLP and QPG generally report more bugs and achieve higher throughput, but at the cost of lower query validity, owing to their efficient metamorphic transformations and broad exploration of query plans. In contrast, DQE exhibits the lowest throughput due to frequent failures during database instance construction; however, it still reports many bugs because it aggressively treats warning inconsistencies across SELECT, UPDATE, and DELETE statements as bugs. Finally, DQP and PQS report fewer bugs, primarily due to their stricter and more conservative oracle validation, which reduces false positives.

As shown in Figure 23 (a)–(d), TiDB reports the largest number of bugs, reaching 381. This is partly due to issues inherent to TiDB itself and partly related to the relatively higher false positive

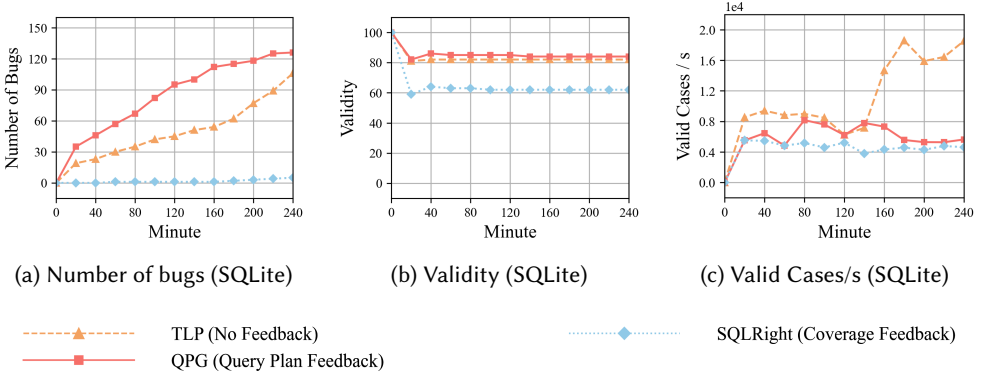


Fig. 24. Comparison of feedback ways for detecting logic bugs.

rates of DQE, TLP, and QPG on this platform. In contrast, CockroachDB reports the fewest bugs, with at most six. However, this does not imply that CockroachDB is more stable; rather, the result is largely due to compatibility issues with DQE and NoREC, which frequently cause failures during database instance construction and significantly reduce the number of valid queries.

Regarding the validity shown in Figures 23 (e)–(h), all fuzzers achieve the highest query validity on MySQL, approaching 100%. This is mainly because MySQL adopts a relatively standard SQL dialect, allowing the generators of different methods to adapt well. For most fuzzers, regardless of the underlying DBMS, validity remains stable between 80% and 100% throughout the evaluation, indicating that existing generators are generally robust, though there is still room for improvement. Since validity is computed cumulatively, the metric gradually stabilizes as execution proceeds. One exception is NoREC on CockroachDB, where validity is noticeably lower due to frequent errors caused by version incompatibilities.

Regarding the valid cases per second metric shown in Figures 23 (i)–(l), all methods achieve substantially higher throughput on SQLite, reaching up to 69,642 valid cases per second. Isolated tests indicate that query execution speed itself is not the primary bottleneck for achieving high throughput. Since all fuzzers connect via JDBC, SQLite’s embedded architecture and extremely low system overhead provide a significant advantage in connection handling. DQE and NoREC achieve only about one case per second on CockroachDB because of compatibility. In contrast, PQS exhibits high throughput because its oracle adopts a partial validation strategy, checking only row-count consistency rather than full result equivalence.

The feedback mechanisms currently applied to logic bug detection mainly include coverage feedback and query plan feedback. We conducted comparative experiments using TLP, QPG, and SQLRight on another SQLite version (3.28.0). These three methods employ the same oracle: TLP does not incorporate feedback, QPG adopts query plan feedback, and SQLRight utilizes coverage feedback. In addition, TLP and QPG share the same generation-based statement generator.

As shown in Figure 24, the results indicate that fuzzing is highly sensitive to the SQLite version, leading to substantial differences across versions. Mismatches between the fuzzer adaptation and the tested DBMS may introduce syntax incompatibilities and even result in a large number of false positives. Within 240 minutes, QPG detects bugs more efficiently than TLP, and the number of detected bugs gradually converges in the later stage, demonstrating that query plan feedback effectively improves the efficiency of logic bug detection. In contrast, SQLRight detects only five bugs; this outcome cannot be solely attributed to coverage feedback, but is largely due to the low efficiency and poor validity of its mutation-based generator. The impact of coverage feedback on

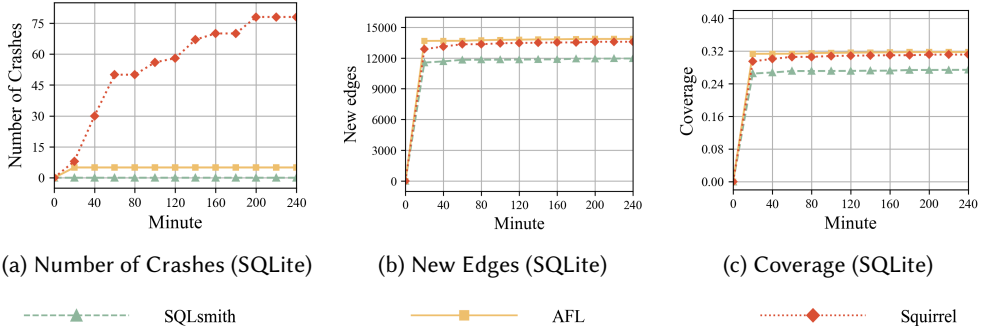


Fig. 25. Comparison of fuzzers for detecting crashes

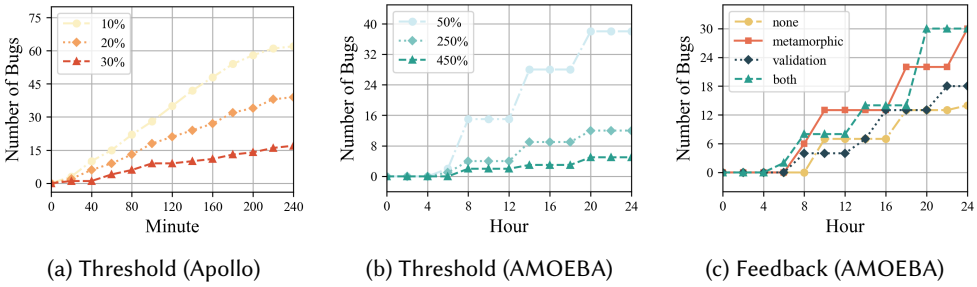


Fig. 26. Evaluation of fuzzers for detecting performance bugs

logic bug detection remains to be studied further. Meanwhile, as expected, introducing feedback slightly reduces the number of valid test cases generated per second.

6.6 Crash Detection Comparison

We tested the performance of Squirrel, AFL, and SQLsmith in detecting crashes on SQLite 3.28.0. Among them, Squirrel is a variant of AFL with improved semantics, and they both utilize coverage feedback. In contrast, SQLsmith does not have a feedback module. As shown in Figure 25, Squirrel can detect the highest number of crashes in the given time range. This indicates that Squirrel's IR-based mutation method is effective in improving crash detection. However, Squirrel does not show a significant improvement in coverage, probably because the semantically incorrect statements generated by AFL also cover a large portion of the DBMS code branches. In summary, increasing coverage does not necessarily improve crash detection efficiency, as only semantically correct statements contribute to crash detection.

6.7 Performance Bugs Detection Evaluation

Although both Apollo and AMOEBA can detect performance bugs, their focuses are different. Apollo focuses on detecting performance degradation caused by version updates, while AMOEBA is used to detect performance bugs caused by insufficient query optimization. Therefore, we chose two PostgreSQL versions 9.6 and 11.2, to evaluate Apollo and tested AMOEBA on the newer of the two versions, PostgreSQL 11.2.

In Figure 26, the experimental results show that both tools detected certain performance bugs in PostgreSQL. It is evident from Figure 26 (a) that in Apollo's experiment, the performance gap between the two versions did not exceed 30% for most test cases, accounting for approximately

63.9%. Figure 26 (b) shows that in AMOEBA, the gap between most equivalent queries is less than 450%, accounting for approximately 86.8%. We analyzed the most severe performance bugs detected by both tools. The results reveal that the performance gap for the most severe bug detected by Apollo is 3.7 times, while for AMOEBA, it is 55.8 times. In general, performance issues caused by query optimization are more serious than those caused by version updates. Figure 26 (c) shows that metamorphic feedback enhances the efficiency of performance bug detection, as it guides the generator to produce more equivalent queries. Compared to no feedback, validation feedback slightly enhances detection capability, but its performance weakens within 10 to 14 minutes. Simultaneously, the combination of metamorphic feedback and validation feedback is less effective than using metamorphic feedback alone. Therefore, it is rational to suspect that validation feedback may not be as valuable as originally expected for the detection of performance bugs. This may be because, although validation feedback can help the generator learn the queries characteristics that trigger bugs, it also tends to generate repeated cases.

7 Conclusion and Challenges

DBMS is the fundamental software for managing and organizing data, and its internal structure is extremely complex. The development of commercial DBMSs often introduces a large number of unexpected bugs (more than 10,000 bugs have been discovered and resolved in the MySQL forum [11]), which can easily lead to huge economic losses. Fuzzing is a method that detects bugs by automatically generating, mutating, and executing test cases. Although fuzzing has been studied for nearly five decades, existing surveys primarily target *general-purpose* software. DBMS contains unique modules and functionalities, but there is no review to deeply study and compare these special DBMS fuzzing methods. This article presents a comprehensive survey of DBMS fuzzing methods. We summarize core concepts, formalize the general fuzzing workflow, and classify existing approaches both horizontally and vertically. Additionally, it reviews recent non-relational DBMS fuzzing methods. The article makes an effort to collect and organize available open-source DBMS fuzzers and conducts comparisons using the same experimental settings to help researchers understand the strengths, weaknesses, and suitability of various methods.

Despite recent progress, DBMS fuzzing still faces challenges in efficiency and generalization. Future research should emphasize component-oriented testing and improved constraint solving, while reducing maintenance costs through more accurate bug reporting. Adapting fuzzing to emerging DBMS types and extending exploration toward semantic search spaces are also critical. Addressing conceptual limits like search space explosion will likely require intelligent guidance and standardization. A detailed discussion of these issues is provided in Appendix B.

References

- [1] 1996. TPC-H. Retrieved March 20, 2026 from <https://www.tpc.org>
- [2] 2011. Data science revealed: A data-driven glimpse into the burgeoning new field. Retrieved March 20, 2026 from <https://www.predictiveanalyticsworld.com/machinelearningtimes/august12/emc-data-science-study.pdf>
- [3] 2015. AFL: American Fuzzy Lop. Retrieved March 20, 2026 from <http://lcamtuf.coredump.cx/afl/>
- [4] 2015. SQLsmith. Retrieved March 20, 2026 from <https://github.com/anse1/sqlsmith>
- [5] 2016. Honggfuzz. Retrieved March 20, 2026 from <https://google.github.io/honggfuzz/>
- [6] 2016. OSS-Fuzz: Continuous Fuzzing for Open Source Software. Retrieved March 20, 2026 from <https://github.com/google/oss-fuzz>
- [7] 2017. LibFuzzer - A Library For Coverage-guided Fuzz Testing. Retrieved March 20, 2026 from <http://llvm.org/docs/LibFuzzer.html>
- [8] 2021. CockroachDB Performance Bug Reports. Retrieved March 20, 2026 from <https://github.com/cockroachdb/cockroach/issues?q=performance>
- [9] 2021. PostgreSQL Performance Bug Reports. Retrieved March 20, 2026 from <https://www.postgresql.org/search/?m=1&q=performance&l=8&d=-1&s=r>

- [10] 2021. SCOTT schema. Retrieved March 20, 2026 from <https://www.orafaq.com/wiki/SCOTT>
- [11] 2023. MySQL Bugs Home. Retrieved March 20, 2026 from <https://bugs.mysql.com/>
- [12] Shadi Abdul Khalek and Sarfraz Khurshid. 2010. Automated SQL query generation for systematic testing of database engines. In *Proceedings of the International Conference on Automated Software Engineering*. ACM, 329–332.
- [13] Jinsheng Ba and Manuel Rigger. 2023. Testing database engines via query plan guidance. In *Proceedings of the International Conference on Software Engineering*. ACM, 2060–2071.
- [14] Jinsheng Ba and Manuel Rigger. 2024. Cert: Finding performance issues in database systems through the lens of cardinality estimation. In *Proceedings of the International Conference on Software Engineering*. 1–13.
- [15] Jinsheng Ba and Manuel Rigger. 2024. Keep it simple: Testing databases via differential query plans. *Proceedings of the ACM on Management of Data* 2, 3 (2024), 1–26.
- [16] Hardik Bati, Leo Giakoumakis, Steve Herbert, and Aleksandras Surna. 2007. A genetic approach for random testing of database systems. In *Proceedings of the International Conference on Very Large Data Bases*. VLDB Endowment, 1243–1251.
- [17] B. Beizer. 1992. Software testing techniques. *Software Testing Verification and Reliability* 2, 4 (1992), 215–216.
- [18] Donald A. Berry and Bert Fristedt. 1985. Bandit problems: Sequential allocation of experiments (Monographs on statistics and applied probability). London: Chapman and Hall 5, 71–87 (1985), 7–7.
- [19] Tim Blazytko, Matt Bishop, Cornelius Aschermann, Justin Cappos, Moritz Schlögel, Nadia Korshun, Ali Abbasi, Marco Schweighauser, Sebastian Schinzel, Sergej Schumilo, et al. 2019. GRIMOIRE: Synthesizing structure while fuzzing. In *Proceedings of the USENIX Security Symposium*. USENIX Association, 1985–2002.
- [20] Nicolas Bruno and Surajit Chaudhuri. 2005. Flexible database generators. In *Proceedings of the International Conference on Very Large Data Bases*. VLDB Endowment, 1097–1107.
- [21] N. Bruno, S. Chaudhuri, and D. Thomas. 2006. Generating queries with cardinality constraints for DBMS testing. *IEEE Transactions on Knowledge and Data Engineering* 18, 12 (2006), 1721–1725.
- [22] Marcel Böhme, Van-Thuan Pham, Manh-Dung Nguyen, and Abhik Roychoudhury. 2017. Directed greybox fuzzing. In *Proceedings of the Conference on Computer and Communications Security*. ACM, 2329–2344.
- [23] Marcel Böhme, Van-Thuan Pham, and Abhik Roychoudhury. 2016. Coverage-based greybox fuzzing as markov chain. In *Proceedings of the Conference on Computer and Communications Security*. ACM, 1032–1043.
- [24] Tsong Y. Chen, Shing C. Cheung, and Shiu Ming Yiu. 2020. Metamorphic testing: A new approach for generating next test cases. arXiv:2002.12543. Retrieved from <https://arxiv.org/abs/2002.12543>
- [25] Xinyue Chen, Chenglong Wang, and Alvin Cheung. 2020. Testing query execution engines with mutations. In *Proceedings of the Workshop on Testing Database Systems*. ACM, 1–5.
- [26] Shumo Chu, Chenglong Wang, Konstantin Weitz, and Alvin Cheung. 2017. Cosette: An automated prover for SQL. In *Proceedings of the Conference on Innovative Data Systems Research*.
- [27] Ziyu Cui, Wensheng Dou, Qianwang Dai, Jiansen Song, Wei Wang, Jun Wei, and Dan Ye. 2022. Differentially testing database transactions for fun and profit. In *Proceedings of the International Conference on Automated Software Engineering*. ACM, 1–12.
- [28] Ziyu Cui, Wensheng Dou, Yu Gao, Rui Yang, Yingying Zheng, Jiansen Song, Yuan Feng, and Jun Wei. 2025. Simple testing can expose most critical transaction bugs: Understanding and detecting write-specific serializability violations in database systems. In *Proceedings of the VLDB Endowment*.
- [29] Wenqian Deng, Jie Liang, Zhiyong Wu, Jingzhou Fu, Mingzhe Wang, and Yu Jiang. 2025. CONI: Detecting database connector bugs via state-aware test case generation. In *Proceedings of the International Conference on Software Engineering*. 667–678.
- [30] Wenjing Deng, Qiuyang Mang, Chengyu Zhang, and Manuel Rigger. 2024. Finding logic bugs in spatial database engines via affine equivalent inputs. *Proceedings of the ACM on Management of Data* 2, 6 (2024), 1–26.
- [31] Wensheng Dou, Ziyu Cui, Qianwang Dai, Jiansen Song, Dong Wang, Yu Gao, Wei Wang, Jun Wei, Lei Chen, Hanmo Wang, et al. 2023. Detecting isolation bugs via transaction oracle construction. In *Proceedings of the International Conference on Software Engineering*. IEEE.
- [32] Cundi Fang, Jie Liang, Zhiyong Wu, Jingzhou Fu, Zhouyang Jia, Chun Huang, Yu Jiang, and Shanshan Li. 2025. DepState: Detecting synchronization failure bugs in distributed database management systems. *Proceedings of the ACM on Software Engineering* 2, ISSTA (2025), 2001–2022.
- [33] Jingzhou Fu, Jie Liang, Zhiyong Wu, Mingzhe Wang, and Yu Jiang. 2022. Griffin: Grammar-free DBMS fuzzing. In *Proceedings of the International Conference on Automated Software Engineering*. ACM, 1–12.
- [34] Jingzhou Fu, Jie Liang, Zhiyong Wu, Yanyang Zhao, Shanshan Li, and Yu Jiang. 2025. Understanding and detecting sql function bugs: Using simple boundary arguments to trigger hundreds of DBMS bugs. In *Proceedings of the European Conference on Computer Systems*. 1061–1076.
- [35] César A. Galindo-Legaria, Stefano Stefani, and Florian Waas. 2004. Query processing for SQL updates. In *Proceedings of the International Conference on Management of Data*. ACM, 844–849.

- [36] Xiyue Gao, Zhuang Liu, Yiran Shen, Hui Li, Yingfan Liu, Hongjun Xiao, Yanguo Peng, and Jiangtao Cui. 2025. Fucci: Database transaction fuzzing via random conflict construction and multilevel constraint solving. *Proceedings of the VLDB Endowment* 18, 6 (2025), 1879–1891.
- [37] Leo Giakoumakis and César A Galindo-Legaria. 2008. Testing SQL server’s query optimizer: Challenges, techniques and experiences. *Data Engineering Bulletin* 31, 1 (2008), 36–43.
- [38] Patrice Godefroid, Adam Kiezun, and Michael Y. Levin. 2008. Grammar-based whitebox fuzzing. In *Proceedings of the Conference on Programming Language Design and Implementation*. ACM, 206–215.
- [39] Eugene Goldberg and Yakov Novikov. 2007. BerkMin: A fast and robust SAT-solver. *Discrete Applied Mathematics* 155, 12 (2007), 1549–1561.
- [40] Goetz Graefe. 1993. Query evaluation techniques for large databases. *ACM Computing Surveys* 25, 2 (1993), 73–169.
- [41] Goetz Graefe. 1995. The cascades framework for query optimization. *Data Engineering Bulletin* 18, 3 (1995), 19–29.
- [42] Jim Gray, Prakash Sundaresan, Susanne Englert, Ken Baclawski, and Peter J. Weinberger. 1994. Quickly generating billion-record synthetic databases. In *Proceedings of the International Conference on Management of Data*. ACM, 243–252.
- [43] Zhongxian Gu, Mohamed A. Soliman, and Florian M. Waas. 2012. Testing the accuracy of query optimizers. In *Proceedings of the Workshop on Testing Database Systems*. ACM, 1–6.
- [44] Zongyin Hao, Quanfeng Huang, Chengpeng Wang, Jianfeng Wang, Yushan Zhang, Rongxin Wu, and Charles Zhang. 2023. Pinolo: Detecting logical bugs in database management systems with approximate query synthesis. In *Proceedings of the USENIX Annual Technical Conference*. 345–358.
- [45] Kenneth Houkjaer, Kristian Torp, and Rico Wind. 2006. Simple and realistic data generation. In *Proceedings of the International Conference on Very Large Data Bases*. VLDB Endowment, 1243–1246.
- [46] William E. Howden. 1978. Theoretical and empirical studies of program testing. *IEEE Transactions on Software Engineering* SE-4, 4 (1978), 293–298.
- [47] Jiaxin Hu and Rongxin Wu. 2025. SQLaw: Detecting bugs in GPU database management systems via rule-based differential execution. *IEEE Transactions on Software Engineering* 51, 7 (2025), 2144–2160.
- [48] Toshihide Ibaraki and Tiko Kameda. 1984. On the optimal nesting order for computing N-Relational joins. *ACM Transactions on Database Systems* 9, 3 (1984), 482–502.
- [49] Daniel Jackson. 2002. Alloy: A lightweight object modelling notation. *ACM Transactions on Software Engineering and Methodology* 11, 2 (2002), 256–290.
- [50] Yu Jiang, Jie Liang, Fuchen Ma, Yuanliang Chen, Chijin Zhou, Yuheng Shen, Zhiyong Wu, Jingzhou Fu, Mingzhe Wang, Shanshan Li, et al. 2024. When fuzzing meets llms: Challenges and opportunities. In *Proceedings of the ACM International Conference on the Foundations of Software Engineering*. 492–496.
- [51] Yuancheng Jiang, Jiahao Liu, Jinsheng Ba, Roland H. C. Yap, Zhenkai Liang, and Manuel Rigger. 2024. Detecting logic bugs in graph database management systems via injective and surjective graph query transformation. In *Proceedings of the International Conference on Software Engineering*. 1–12.
- [52] Yuancheng Jiang, Jianing Wang, Chuqi Zhang, Roland Yap, Zhenkai Liang, and Manuel Rigger. 2025. Enhanced differential testing in emerging database systems. arXiv:2501.01236. Retrieved from <https://arxiv.org/abs/2501.01236>
- [53] Zu-Ming Jiang, Jia-Ju Bai, and Zhendong Su. 2023. DynSQL: Stateful fuzzing for database management systems with complex and valid SQL query generation. In *Proceedings of the USENIX Security Symposium*. USENIX Association, 4949–4965.
- [54] Zu-Ming Jiang, Si Liu, Manuel Rigger, and Zhendong Su. 2023. Detecting transactional bugs in database engines via {graph-based} oracle construction. In *Proceedings of the USENIX Symposium on Operating Systems Design and Implementation*. 397–417.
- [55] Zu-Ming Jiang and Zhendong Su. 2024. Detecting logic bugs in database engines via equivalent expression transformation. In *Proceedings of the USENIX Symposium on Operating Systems Design and Implementation*. 821–835.
- [56] Jinho Jung, Hong Hu, Joy Arulraj, Taesoo Kim, and Woonhak Kang. 2019. APOLLO: Automatic detection and diagnosis of performance regressions in database systems. *Very Large Data Bases Endowment* 13, 1 (2019), 57–70.
- [57] Matteo Kamm, Manuel Rigger, Chengyu Zhang, and Zhendong Su. 2023. Testing graph database engines via query partitioning. In *Proceedings of the ACM SIGSOFT International Symposium on Software Testing and Analysis*.
- [58] Shadi Abdul Khalek, Bassem Elkarablieh, Yai O. Laleye, and Sarfraz Khurshid. 2008. Query-aware test generation using a relational constraint solver. In *Proceedings of the International Conference on Automated Software Engineering*. IEEE, 238–247.
- [59] Sarfraz Khurshid and Darko Marinov. 2004. TestEra: Specification-based testing of Java programs using SAT. *Automated Software Engineering* 11, 4 (2004), 403–434.
- [60] Donghun Lee, Sang K. Cha, and Arthur H. Lee. 2011. A performance anomaly detection and analysis framework for DBMS development. *IEEE Transactions on Knowledge and Data Engineering* 24, 8 (2011), 1345–1360.

- [61] Viktor Leis, Andrey Gubichev, Atanas Mirchev, Peter Boncz, Alfons Kemper, and Thomas Neumann. 2015. How good are query optimizers, really? *Very Large Data Bases Endowment* 9, 3 (2015), 204–215.
- [62] Caroline Lemieux and Koushik Sen. 2018. FairFuzz: A targeted mutation strategy for increasing greybox fuzz testing coverage. In *Proceedings of the International Conference on Automated Software Engineering*. ACM, 475–485.
- [63] Jiaqi Li, Ke Wang, Yaoguang Chen, Yajin Zhou, Lei Wu, and Jiashui Wang. 2023. Detecting dbms bugs with context-sensitive instantiation and multi-plan execution. arXiv:2312.04941. Retrieved from <https://arxiv.org/abs/2312.04941>
- [64] Jiaqi Li, Yajin Zhou, and Lei Wu. 2024. Improving multitasking DBMS fuzzing With more accurate coverage and testcase trimming. *IEEE Transactions on Dependable and Secure Computing* 22, 3 (2024), 2756–2770.
- [65] Zhan Li, Olga Papaemmanouil, and Mitch Cherniack. 2016. Optmark: A Toolkit for Benchmarking Query Optimizers. In *Proceedings of the Information and Knowledge Management*. ACM, 2155–2160.
- [66] Jie Liang, Yaoguang Chen, Zhiyong Wu, Jingzhou Fu, Mingzhe Wang, Yu Jiang, Xiangdong Huang, Ting Chen, Jiashui Wang, and Jiajia Li. 2023. Sequence-oriented DBMS fuzzing. In *Proceedings of the International Conference on Data Engineering*. IEEE.
- [67] Jie Liang, Zhiyong Wu, Jingzhou Fu, Mingzhe Wang, Chengnian Sun, and Yu Jiang. 2024. Mozi: Discovering dbms bugs via configuration-based equivalent transformation. In *Proceedings of the International Conference on Software Engineering*. 1–12.
- [68] Yu Liang, Song Liu, and Hong Hu. 2022. Detecting logical bugs of DBMS with coverage-based guidance. In *Proceedings of the USENIX Security Symposium*. USENIX Association, 4309–4326.
- [69] Li Lin, Qinglin Zhu, Hongqiao Chen, Zhuangda Wang, Rongxin Wu, and Xiaoheng Xie. 2025. QTRAN: Extending metamorphic-oracle based logical bug detection techniques for multiple-DBMS dialect support. *Proceedings of the ACM on Software Engineering* 2, ISSTA (2025), 731–752.
- [70] Ziyue Hua, Wei Lin, Luyao Ren, Zongyang Li, Lu Zhang, Wenpin Jiao, and Tao Xie. 2023. GDsmith: Detecting bugs in cypher graph database engines. In *Proceedings of the International Symposium on Software Testing and Analysis*. 163–174.
- [71] Shuang Liu, Junhao Lan, Xiaoning Du, Jiyuan Li, Wei Lu, Jiajun Jiang, and Xiaoyong Du. 2024. Testing graph database systems with graph-state persistence oracle. In *Proceedings of the ACM SIGSOFT International Symposium on Software Testing and Analysis*. 666–677.
- [72] Xinyu Liu, Qi Zhou, Joy Arulraj, and Alessandro Orso. 2022. Automatic detection of performance bugs in database systems using equivalent queries. In *Proceedings of the International Conference on Software Engineering*. ACM, 225–236.
- [73] Eric Lo, Carsten Binnig, Donald Kossmann, M. Tamer Özsu, and Wing-Kai Hon. 2010. A framework for testing DBMS features. *Very Large Data Bases Endowment* 19, 2 (2010), 203–230.
- [74] Ce Lyu, Minghao Zhao, Yanhao Wang, and Liang Jie. 2025. LLM-based dynamic differential testing for database connectors with reinforcement learning-guided prompt election. In *Proceedings of the International Conference on Automated Software Engineering*. ACM, 3814–3818.
- [75] Qiuyang Mang, Jinsheng Ba, Pinjia He, and Manuel Rigger. 2025. Finding logic bugs in graph-processing systems via graph-cutting. *Proceedings of the ACM on Management of Data* 3, 3 (2025), 1–27.
- [76] Qiuyang Mang, Aoyang Fang, Boxi Yu, Hanfei Chen, and Pinjia He. 2024. Testing graph database systems via equivalent query rewriting. In *Proceedings of the International Conference on Software Engineering*.
- [77] Darko Marinov. 2010. Session Details: Technical Session 8: Concurrency and differential testing. In *Proceedings of the International Symposium on Software Testing and Analysis*. ACM.
- [78] William M. McKeeman. 1998. Differential testing for software. *Digital Technical Journal* 10, 1 (1998), 100–107.
- [79] Phil McMinn, Chris J. Wright, and Gregory M. Kapfhammer. 2015. The effectiveness of test coverage criteria for relational database schema integrity constraints. *ACM Transactions on Software Engineering and Methodology* 25, 1 (2015), 1–49.
- [80] Kaiming Mi, Chunxi Zhang, Weining Qian, and Rong Zhang. 2021. Artemis: An automatic test suite generator for large scale OLAP database. In *Proceedings of the Benchmarking, Measuring, and Optimizing: 3rd BenchCouncil International Symposium*. Springer, 74–89.
- [81] Barton P. Miller, Lars Fredriksen, and Bryan So. 1990. An empirical study of the reliability of UNIX utilities. *Communications of the ACM* 33, 12 (1990), 32–44.
- [82] Chaitanya Mishra, Nick Koudas, and Calisto Zuzarte. 2008. Generating targeted queries for database testing. In *Proceedings of the International Conference on Management of Data*. ACM, 499–510.
- [83] Jayashree Mohan, Ashlie Martinez, Soujanya Ponnappalli, Pandian Raju, and Vijay Chidambaram. 2018. Finding crash-consistency bugs with bounded black-box crash testing. In *Proceedings of the USENIX Symposium on Operating Systems Design and Implementation*. USENIX Association, 33–50.
- [84] MySQL. 2023. The MySQL Test Framework. Retrieved March 20, 2026 from https://dev.mysql.com/doc/dev/mysql-server/latest/PAGE_MYSQL_TEST_RUN.html

- [85] Andrea Neufeld, Guido Moerkotte, and Peter C Loekemann. 1993. Generating consistent test data: Restricting the search space by A generator formula. *Very Large Data Bases Endowment* 2, 2 (1993), 173–213.
- [86] Carlos Pacheco, Shuvendu K. Lahiri, Michael D. Ernst, and Thomas Ball. 2007. Feedback-directed random test generation. In *Proceedings of the International Conference on Software Engineering*. IEEE, 75–84.
- [87] PingCap. 2023. go randgen. Retrieved March 20, 2026 from <https://github.com/pingcap/go-randgen>
- [88] Meikel Poess and John M. Stephens. 2004. Generating thousand benchmark queries in seconds. In *Proceedings of the International Conference on Very Large Data Bases*. VLDB Endowment, 1045–1053.
- [89] PostgreSQL. 2023. Regression Tests. Retrieved March 20, 2026 from <https://www.postgresql.org/docs/current/regress.html>
- [90] Manuel Rigger and Zhendong Su. 2020. Detecting optimization bugs in database engines via non-optimizing reference engine construction. In *Proceedings of the Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. ACM.
- [91] Manuel Rigger and Zhendong Su. 2020. Finding bugs in database systems via query partitioning. *Programming Languages 4*, OOPSLA (2020), 1–30.
- [92] Manuel Rigger and Zhendong Su. 2020. Testing database engines via pivoted query synthesis. In *Proceedings of the USENIX Symposium on Operating Systems Design and Implementation*. USENIX Association, 667–682.
- [93] Manuel Rigger and Zhendong Su. 2022. Intramorphic testing: A new approach to the test oracle problem. In *Proceedings of the International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software*. ACM, 128–136.
- [94] Shetal Shah, S. Sudarshan, Suhas Kajbaje, Sandeep Patidar, Bhanu Pratap Gupta, and Devang Vira. 2011. Generating test data for killing SQL mutants: A constraint-based approach. In *Proceedings of the International Conference on Data Engineering*.
- [95] Donald R. Slutz. 1998. Massive stochastic testing of SQL. In *Proceedings of the International Conference on Very Large Data Bases*. VLDB Endowment, 618–622.
- [96] Jiansen Song, Wensheng Dou, Ziyu Cui, Qianwang Dai, Wei Wang, Jun Wei, Hua Zhong, and Tao Huang. 2023. Testing database systems via differential query execution. In *Proceedings of the International Conference on Software Engineering*.
- [97] Jiansen Song, Wensheng Dou, Yu Gao, Ziyu Cui, Yingying Zheng, Dong Wang, Wei Wang, Jun Wei, and Tao Huang. 2024. Detecting metadata-related logic bugs in database systems via raw database construction. *Proceedings of the VLDB Endowment* 17, 8 (2024), 1884–1897.
- [98] Jiansen Song, Wensheng Dou, Yingying Zheng, Yu Gao, Ziyu Cui, Wei Wang, and Jun Wei. 2025. Detecting schema-related logic bugs in relational dbms via equivalent database construction. *Proceedings of the VLDB Endowment* 18, 7 (2025) 2281–2294.
- [99] SQLite. 2023. How SQLite Is Tested. Retrieved March 20, 2026 from <https://sqlite.org/testing.html>
- [100] Keith Stobie. 2005. Too darned big to test: Testing large systems is a daunting task, but there are steps we can take to ease the pain. *Queue* 3, 1 (2005), 30–37.
- [101] Xiu Tang, Sai Wu, Dongxiang Zhang, Feifei Li, and Gang Chen. 2023. Detecting logic bugs of join optimizations in DBMS. *International Conference on Management of Data* 1, 1 (2023), 1–26.
- [102] Patrick Valduriez and Scott Danforth. 1992. Functional SQL (FSQL), an SQL upward-compatible database programming language. *Information Sciences* 62, 3 (1992), 183–203.
- [103] Junjie Wang, Bihuan Chen, Lei Wei, and Yang Liu. 2019. Superion: Grammar-aware greybox fuzzing. In *Proceedings of the International Conference on Software Engineering*. IEEE, 724–735.
- [104] Mingzhe Wang, Zhiyong Wu, Xinyi Xu, Jie Liang, Chijin Zhou, Huafeng Zhang, and Yu Jiang. 2021. Industry practice of coverage-guided enterprise-level DBMS fuzzing. In *Proceedings of the International Conference on Software Engineering: Software Engineering in Practice*. IEEE, 328–337.
- [105] Shihao Wen, Peng Jia, Pin Yang, and Chi Hu. 2023. Squill: Testing DBMS with correctness feedback and accurate instantiation. *Applied Sciences* 13, 4 (2023), 2519.
- [106] Zhiyong Wu, Jie Liang, Jingzhou Fu, Mingzhe Wang, and Yu Jiang. 2025. Puppy: Finding performance degradation bugs in dbms via limited-optimization plan construction. In *Proceedings of the International Conference on Software Engineering*. 679–690.
- [107] Zhiyong Wu, Jie Liang, Mingzhe Wang, Chijin Zhou, and Yu Jiang. 2022. Unicorn: Detect runtime errors in time-series databases with hybrid input synthesis. In *Proceedings of the International Symposium on Software Testing and Analysis*.
- [108] Jiaqi Yan, Qiuye Jin, Shrainik Jain, Stratis D. Viglas, and Allison Lee. 2018. Snowtrail: Testing with production queries on a cloud database. In *Proceedings of the Workshop on Testing Database Systems*. ACM, 1–6.
- [109] Rui Yang, Yingying Zheng, Lei Tang, Wensheng Dou, Wei Wang, and Jun Wei. 2023. Randomized differential testing of RDF stores. In *International Conference on Software Engineering: Companion Proceedings*. IEEE, 136–140.

- [110] Yupeng Yang, Yongheng Chen, Rui Zhong, Jizhou Chen, and Wenke Lee. 2024. Towards generic database management system fuzzing. In *Proceedings of the USENIX Security Symposium*. 901–918.
- [111] Zijing Yin, Si Liu, and David Basin. 2025. Testing graph databases with synthesized queries. *Proceedings of the ACM on Management of Data* 3, 4 (2025), 1–26.
- [112] Chi Zhang and Manuel Rigger. 2025. Constant optimization driven database system testing. *Proceedings of the ACM on Management of Data* 3, 1 (2025), 1–24.
- [113] Yushan Zhang, Peisen Yao, Rongxin Wu, and Charles Zhang. 2021. Duplicate-sensitivity guided transformation synthesis for DBMS correctness bug detection. arXiv:2107.03660. Retrieved from <https://arxiv.org/abs/2107.03660>
- [114] Yingying Zheng, Wensheng Dou, Lei Tang, Ziyu Cui, Yu Gao, Jiansen Song, Liang Xu, Jiaxin Zhu, Wei Wang, Jun Wei, et al. 2024. Testing gremlin-based graph database systems via query disassembling. In *Proceedings of the ACM SIGSOFT International Symposium on Software Testing and Analysis*. 1695–1707.
- [115] Yingying Zheng, Wensheng Dou, Yicheng Wang, Zheng Qin, Lei Tang, Yu Gao, Dong Wang, Wei Wang, and Jun Wei. 2022. Finding bugs in gremlin-based graph database systems via randomized differential testing. In *Proceedings of the International Symposium on Software Testing and Analysis*. ACM, 302–313.
- [116] Rui Zhong, Yongheng Chen, Hong Hu, Hangfan Zhang, Wenke Lee, and Dinghao Wu. 2020. SQUIRREL: Testing Database Management Systems with Language Validity and Coverage Feedback. In *Proceedings of the Conference on Computer and Communications Security*. ACM, 955–970.
- [117] Suyang Zhong and Manuel Rigger. 2025. Testing Database Systems with Large Language Model Synthesized Fragments. arXiv:2505.02012. Retrieved from <https://arxiv.org/abs/2505.02012>
- [118] Zenghui Zhou, Zheng Zheng, Tsong Yueh Chen, Jinyi Zhou, and Kun Qiu. 2021. Follow-up test cases are better than source test cases in metamorphic testing: A preliminary study. In *Proceedings of the International Workshop on Metamorphic Testing*. IEEE, 69–74.
- [119] Zeyang Zhuang, Penghui Li, Pingchuan Ma, Wei Meng, and Shuai Wang. 2023. Testing graph database systems via graph-aware metamorphic relations. *Proceedings of the VLDB Endowment* 17, 4 (2023), 836–848.

Received 17 November 2023; revised 20 December 2025; accepted 10 February 2026